



Advances and Frontiers of LLM-based Issue Resolution in Software Engineering: A Comprehensive Survey

Caihua Li¹, Lianghong Guo¹, Yanlin Wang^{1*}, Daya Guo¹, Wei Tao^{2*}, Zhenyu Shan³, Mingwei Liu¹, Jiachi Chen⁴, Haoyu Song⁵, Duyu Tang⁵, Hongyu Zhang⁶, Zibin Zheng¹

¹Sun Yat-sen University, ²Independent Researcher, ³Hangzhou Normal University,

⁴Zhejiang University, ⁵Huawei Technologies Co, Ltd, ⁶Chongqing University

{lich535, guolh8, guody5}@mail2.sysu.edu.cn, {wangylin36, zhzhbin}@mail.sysu.edu.cn,

wtao@ieee.org, 20100119@hznu.edu.cn, chenjiachi317@gmail.com,

{songhaoyu1, tangduyu}@huawei.com, hyzhang@cqu.edu.cn

Abstract

Issue resolution, a complex Software Engineering (SWE) task integral to real-world development, has emerged as a compelling challenge for artificial intelligence. The establishment of benchmarks like SWE-bench revealed this task as profoundly difficult for large language models, thereby significantly accelerating the evolution of autonomous coding agents. This paper presents a systematic survey of this emerging domain. We begin by examining data construction pipelines, covering automated collection and synthesis approaches. We then provide a comprehensive analysis of methodologies, spanning training-free frameworks with their modular components to training-based techniques, including supervised fine-tuning and reinforcement learning. Subsequently, we discuss critical analyses of data quality and agent behavior, alongside practical applications. Finally, we identify key challenges and outline promising directions for future research. An open-source repository is maintained at <https://github.com/DeepSoftwareAnalytics/Awesome-Issue-Resolution> to serve as a dynamic resource in this field.

1 Introduction

The vision of constructing a true AI software engineer has long been an appealing prospect in computer science. In pursuit of this, researchers initially relied on function-level code generation benchmarks such as HumanEval (Chen et al., 2021). Driven by the remarkable success of Large Language Models (LLMs), the prospect of automating software engineering to function akin to a human developer has become increasingly attainable. However, they often struggle to handle the dynamic interactions with development environments and human collaboration in realistic scenarios. To address this limitation and align evaluation with authentic software development scenarios, Jimenez

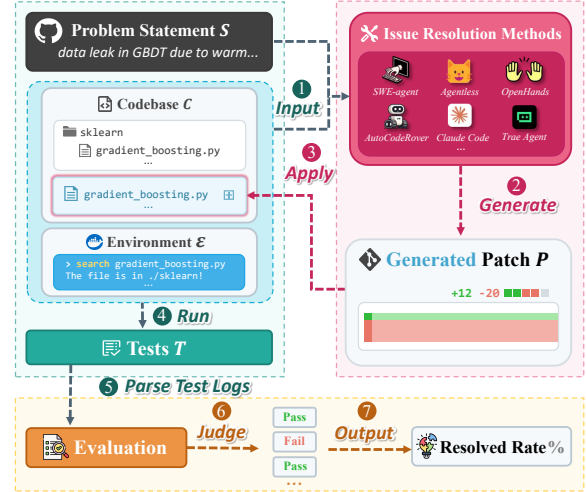


Figure 1: Issue resolution task.

et al. (2024) introduced SWE-bench and defined the task of issue resolution. This task requires an automatic approach to help LLMs navigate complex, multi-file repositories to resolve issues (see Figure 1 and Section 2). By revealing the difficulty of repository-level engineering, SWE-bench catalyzed a research frontier focused on navigating and modifying environments (Pan et al.). It marks a departure from initial software generation as explored in ChatDev (Qian et al., 2024) and MetaGPT (Hong et al., 2024), to the subsequent stages of software maintenance and evolution.

Despite the surge of research in this new frontier, the literature remains fragmented. Current surveys primarily focus on code generation, failing to address the far more complex challenge of issue resolution. This paper aims to bridge this gap by providing the first in-depth survey of this domain.

We conducted a comprehensive survey of publicly available literature, including 175 papers and online resources relevant to issue resolution. We established a tailored classification framework to provide a structured perspective on this rapidly evolving domain. Consequently, our contributions

*Corresponding authors.

can be summarized as follows: We present a **systematic survey** on issue resolution, organized by a **structured taxonomy** covering Data, Methods, and Analysis. Furthermore, we identify **key challenges and future directions**, and provide a **open-source repository** to support the community.

2 Task formulation

Issue resolution requires LLMs to synthesize a valid code change (also called a patch) $\mathcal{P}$ to resolve the issue, interacting with a codebase (as shown in Figure 1). Formally, an instance of the task can be expressed as $\mathcal{I} = (\mathcal{D}, \mathcal{C}, \mathcal{T})$, comprising an issue description $\mathcal{D}$, the codebase $\mathcal{C}$, and corresponding tests $\mathcal{T}$. The observable parts of the instance, $\mathcal{D}$ and $\mathcal{C}$, are available during the resolving process, which contains supplementary information on the corresponding environment $\mathcal{E}$ that can be explored. So a method $\mathcal{M}$ is expected to achieve:

$$\mathcal{P} = \mathcal{M}(\mathcal{D}, \mathcal{C}, \mathcal{E}) \quad (1)$$

After the patch $\mathcal{P}$ is applied to $\mathcal{C}$, the evaluation is conducted through running tests on the modified codebase $\mathcal{C}' = \text{Apply}(\mathcal{C}, \mathcal{P})$. The resolution outcome $r \in \{0, 1\}$ is then determined by the execution of $\mathcal{T}$, denoted as $\text{Exec}(\mathcal{C}', \mathcal{T})$. On a dataset including n instances ($\mathbb{I} = \{\mathcal{I}_i\}_{i=1}^N$), overall performance metric is:

$$\text{Resolved Rate} = \frac{1}{|\mathbb{I}|} \sum_{i=1}^{|\mathbb{I}|} r_i \quad (2)$$

3 Data

Data is fundamental to the issue-resolution task, serving as both an evaluation benchmark and a training resource. Thus, datasets are classified into evaluation (§ 3.1.1) and training (§ 3.1.2) sets. Construction approaches are divided into two types: data collection (§ 3.2.1) from real-world online sources, and data synthesis (§ 3.2.2) achieved by rewriting real-world data or rule-based generation. Statistics for all datasets are provided in § A.3.

3.1 Datasets

3.1.1 Evaluation datasets

SWE-bench (Jimenez et al., 2024) established the datasets by collecting issue–Pull Request (PR) pairs from popular Python repositories, pairing each issue with a full repository snapshot for resolution. However, invalid tests and underspecified

descriptions in the original dataset make many instances unsuitable for evaluation. To ensure data quality, SWE-bench Verified (OpenAI, 2024) was introduced, offering a subset of manually validated samples as a trusted benchmark.

While most evaluation datasets target Python, researchers extend these tasks to ten different programming languages to broaden the linguistic scope, as seen in SWE-bench Multilingual and Multi-SWE-bench, et al. (Zan et al., 2024, 2025; Rashid et al., 2025; Guo et al., 2025b; Yang et al., 2025d; Mhatre et al., 2025). To address the limitation of relying solely on textual data, researchers have focused on aggregating multimodal information, primarily derived from images such as UI screenshots and diagrams (Yang et al., 2025c; Zhang et al., 2025g; Guo et al., 2025b). For instance, Yang et al. (2025c) integrates these visual contexts and introduces a novel visual tester to validate the correctness of visual modifications.

To bridge the gap between initial evaluation datasets and realistic software development scenarios, researchers introduced datasets (Miserendino et al., 2025; Deng et al., 2025a; Tarasova et al., 2025) that incorporate enterprise-level complexity and diverse domains. Furthermore, the scope of issue resolution tasks has been expanded to broader vision on software engineering, such as incremental development (Li et al., 2025c) and long-horizon software evolution (Zeng et al., 2025b). Recent works have also shifted towards refining metrics for issue resolution, such as efficiency and safety (He et al., 2025a; Ma et al., 2025a; Xu et al., 2025a).

3.1.2 Training datasets

Textual data The initial training data typically consists of raw task instances in the form of static issue-PR, as exemplified by the training set in SWE-bench (Jimenez et al., 2024), to equip models with fundamental capabilities to resolve issues before they engage with interactive environments.

Training Environment Environment datasets aim to address the limitations of static text by constructing interactive code environments that provide LLMs with execution feedback. In early initiatives, researchers attempted to equip each task instance with a corresponding Conda or Docker environment, enabling LLMs to incorporate code execution results as feedback during the training process, as seen in benchmarks like Multi-SWE-RL (Zan et al., 2025). Nevertheless, these datasets often

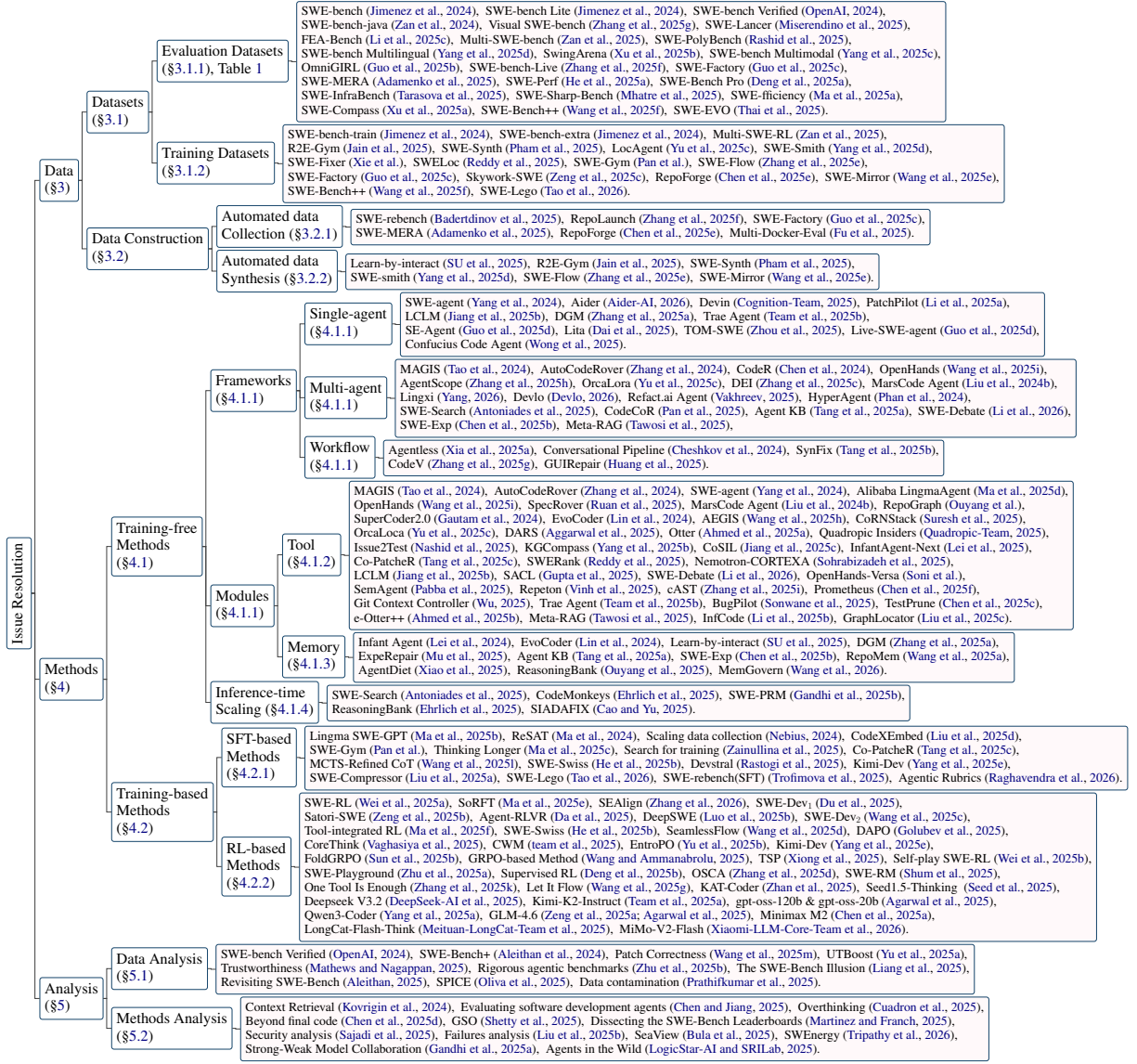


Figure 2: Overall perspective on data, methods, and analysis for SWE tasks, featuring corresponding papers.

overlook the specific interface design required for effective LLM-environment interaction. To address this, Jain et al. (2025) introduced a more interactive Gym environment (R2E-Gym) that utilizes LLM-synthesized test cases to verify environment usability, thereby synthesizing large-scale environment data. Similarly, Pan et al. constructed Gym environments based on real-world GitHub issues.

Trajectories Trajectory datasets capture the procedural interplay between LLMs and execution environments through tool invocations and feedback loops (Nebius, 2024; Jain et al., 2025; Pan et al.; Pham et al., 2025; Xie et al.; Guo et al., 2025c)(more details in § A.3). To obtain high-quality trajectories, researchers typically employ inference-time scaling strategies to generate many candidates, then apply verifiers to filter and select the best (Nebius, 2024; Yang et al., 2025d).

3.2 Data construction

Training data construction for software agents is transitioning from manual processes to automated pipelines (see § A.2 for background details).

3.2.1 Automated data collection

Static datasets often suffer from rigidity, high maintenance costs, and limited scale, which impede the effective training of robust models. In response, the field is evolving towards scalable, automated data collection methods. Those automated pipelines usually leverage LLMs to explore repository configurations, identify files related to environment setup, and generate corresponding dependency installation commands to build Docker images for individual issues. Subsequently, they employ existing testing frameworks and predefined log parsers to analyze test execution results, encompassing both workflow-based and agent-based paradigms, such

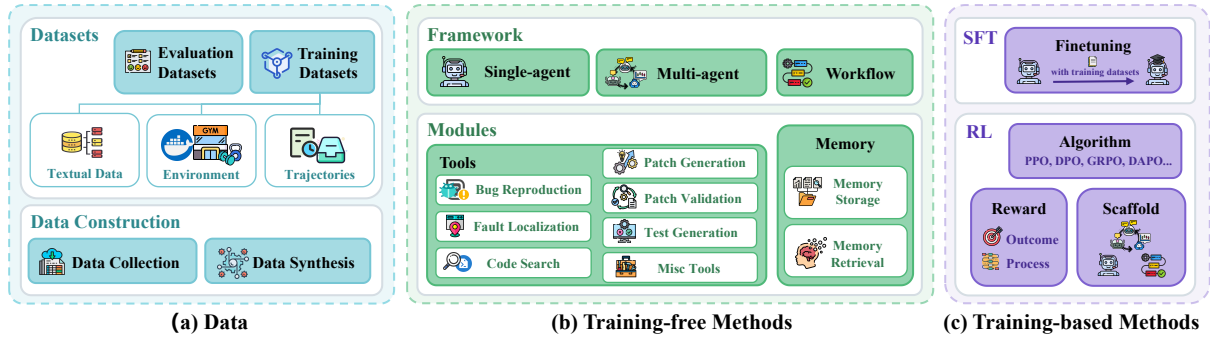


Figure 3: A classification overview of data, training-free methods, and training methods for solving SWE tasks.

as SWE-rebench and RepoLaunch (Badertdinov et al., 2025; Zhang et al., 2025f). Notably, SWE-Factory employs a memory-enhanced framework for environment setup and verification, while utilizing an exit-code-based automatic grading method to design parsers capable of automatically interpreting execution results across diverse testing frameworks in different programming languages (Guo et al., 2025c). More recently, RepoForge (Chen et al., 2025e) enhanced the pipeline’s automation by incorporating a automatic verification mechanism based on SPICE (Oliva et al., 2025) following data construction, effectively replacing the need for human expert validation.

3.2.2 Automated data synthesis

To address limited real-world data and costly manual verification, researchers increasingly adopted automated data synthesis approaches. For instance, SWE-Synth (Pham et al., 2025) rewrites target code and generates corresponding tests. Drawing on test-driven development, SWE-Flow (Zhang et al., 2025e) employs a runtime dependency graph to derive incremental code and requirements from unit tests. More recently, SWE-Smith (Yang et al., 2025d) scales data by paraphrasing descriptions and injecting bugs, leveraging a shared environment to reduce storage overhead. Similarly, SWE-Mirror (Wang et al., 2025e) transplants real-world issues into target repositories to generate verifiable tasks under shared environments.

4 Methods

4.1 Training-free method

To overcome constraints such as fixed context windows and static knowledge, training-free methods utilizes external components and sophisticated prompting. As shown in Figure 3, we classify these methods into three categories based on the

underlying framework: (1) *Frameworks*, encompassing high-level architectures like single-agent, multi-agent, and fixed-workflow designs; (2) *Modules*, providing plug-and-play augmentations such as *Tools* for repository interaction and *Memory* for experience accumulation; and (3) *Inference-time Scaling* (or test-time scaling), employing search or parallelization strategies to enhance success rates without modifying model parameters.

4.1.1 Frameworks

To handle the multi-stage execution required for issue resolution, current research structures LLM activities into either dynamic agent-based or rigid workflow-based frameworks.

Single-agent Analogous to software engineers who write code and invoke diverse tools to resolve issues, single-agent frameworks were initially constructed to execute tasks via tool-based interaction paradigms. SWE-agent (Yang et al., 2024) pioneers the agent-computer interface, which enables autonomous file navigation, code editing, and test execution, bridging natural language understanding with repository-level operations.

However, granting full autonomy for every decision often leads to redundant action sequences due to reasoning imprecision, resulting in prohibitive operational costs. To address it, Li et al. (2025a) reduces overhead by either constraining specific phases into rigid processes.

To further enhance generalization across diverse issue types, self-evolutionary frameworks have emerged to autonomously refine agent capabilities. For instance, Darwin Gödel Machine employs an evolutionary process starting from a minimal baseline, where the LLM generates, scores, and selects optimal candidate agent implementations over successive iterations to evolve its structure (Zhang et al., 2025a; Xia et al., 2025b; Guo et al., 2025d).

Multi-agent Introduced concurrently with single-agent systems, multi-agent frameworks focus on collaboration and task allocation, often performed in the form of human software development team (Tao et al., 2024; Zhang et al., 2024; Liu et al., 2024b; Antoniadou et al., 2025; Yu et al., 2025c). For instance, MAGIS (Tao et al., 2024) firstly implements this by assigning four agents customized for software evolution, enabling role-playing and autonomous meetings for effective communication.

However, those works largely relies on text-based contexts for information exchange and lacks explicit modeling of agent collaboration. To address this, CodeR (Chen et al., 2024) introduces task graphs that convert a high-level plan into a parsable, directed graph to ensure precise execution. Similarly, SWE-Debate (Li et al., 2026) adopts graph-based structures to orchestrate a three-round debate among specialized agents along code dependency traces, yielding more concrete solutions.

With the proliferation of diverse agent frameworks, recent research has shifted towards unified platforms capable of orchestrating collaboration among heterogeneous agents (Wang et al., 2025i; Zhang et al., 2025c). For example, Zhang et al. (2025c) proposed DEIBase, which leverages LLMs to score and rank solutions generated by multiple agents, achieving superior performance over single-agent approaches.

Workflow Workflow architectures improve stability by enforcing predefined steps instead of open-ended exploration. Xia et al. (2025a) adopted a linear pipeline (localization, repair, and validation) to ensure efficiency and reproducibility. For visual tasks, researchers use vision language models to convert UI screenshots into code (Huang et al., 2025) or textual descriptions (Zhang et al., 2025g). To handle complex codebases, Tang et al. (2025b) utilized dependency graphs to guide precise, repository-wide modifications instead of random search.

4.1.2 Tool modules

In training-free frameworks, LLMs rely on specialized tools to augment reasoning without fine-tuning. These tools are organized by the standard repair pipeline, progressing from bug reproduction, fault localization, and code search to patch generation, validation, and test generation (See Figure 4).

Bug reproduction tools. These tools automate the critical first step of debugging by generating

executable scripts that trigger reported defects. Implementations typically leverage historical interaction data to adapt to repository-specific conventions (Lin et al., 2024), or employ finite state machines to govern behavior via multi-dimensional feedback, as in AEGIS (Wang et al., 2025h).

Fault localization tools. Once a bug is reproduced, these tools pinpoint suspicious code regions to narrow the search space. Common approaches include method-level Spectrum-Based Fault Localization (SBFL) (Zhang et al., 2024) and graph-based methods that construct code dependency graphs to trace fault propagation (Li et al., 2026).

Code search tools. These tools retrieve relevant dependency context after localization. Strategies range from interactive retrieval using BM25 or AST-based APIs (Tao et al., 2024; Yang et al., 2024), to graph-based global understanding via Knowledge Graphs and Language Server Protocols (Ma et al., 2025d; Liu et al., 2024b), and dynamic managers that balance exploration breadth and depth (Yu et al., 2025c; Jiang et al., 2025c).

Patch generation tools. These tools enhance LLM output quality through structured methodologies, including augmenting input context via specification inference (Ruan et al., 2025), employing robust editing formats such as AutoDiff to bypass line-numbering failures (Yang et al., 2024; Liu et al., 2024b), and employing ensemble selection mechanisms that filter candidates through regression testing (Team et al., 2025b).

Patch validation tools. These tools confirm correctness and prevent regressions through external verification. Standard approaches include dynamic execution orchestration using sandboxed environments (Zhang et al., 2024; Liu et al., 2024b), and static analysis mechanisms leveraging QA agents or Language Server Protocols for immediate diagnostic feedback (Tao et al., 2024; Liu et al., 2024b).

Test generation tools. These tools generate reproduction test cases to validate intent and guide resolution. Systems typically employ feedback-driven iterative mechanisms that utilize error classification to synthesize failing tests reproducing reported defects, as in Otter (Ahmed et al., 2025a) and Issue2Test (Nashid et al., 2025).

Other extensions. Recent extensions focus on equipping agents with versatile tools to handle

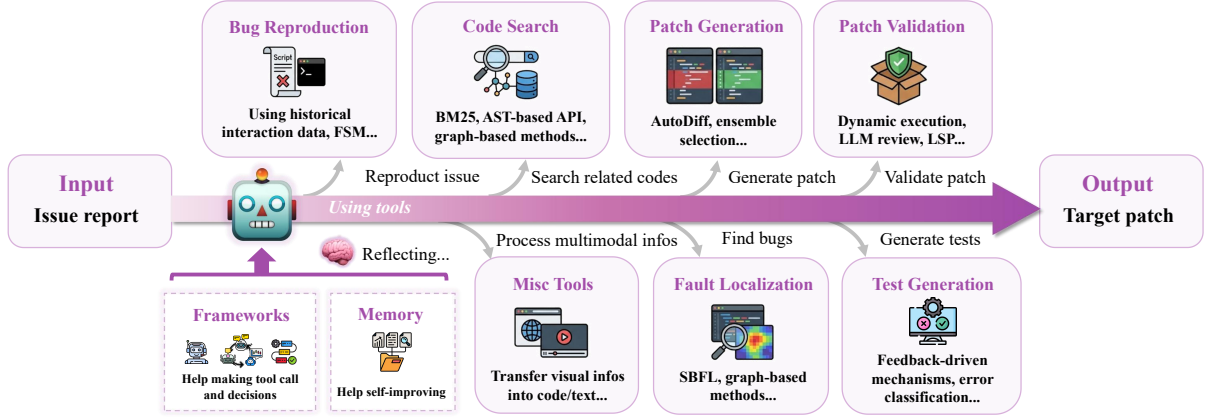


Figure 4: Taxonomy of tool modules for LLM-based issue resolution.

broader scenarios, including multimodal challenges. Strategies involve multimodal browsing and unified information access, which standardizes heterogeneous data into Markdown for seamless processing (Soni et al.; Lei et al., 2025).

4.1.3 Memory modules

Memory integration empowers agents to transcend isolated problem-solving by accumulating historical context to guide future actions. Initial architectures focused on establishing hierarchical storage structures, such as segregating general knowledge from repository-specific details to mitigate rigidity (Lin et al., 2024), or archiving populations of agent variants to support open-ended evolution (Zhang et al., 2025a). To overcome the limitations of static prompting, Mu et al. (2025) incorporated dual-process cognitive architectures that synergize episodic records of concrete repairs with semantic layers of abstract insight, enabling dynamic retrieval based on current context. Current frontiers prioritize distilling transferable reasoning strategies, effectively shifting the paradigm from storing raw data to abstracting high-level policies from both successful and failed trajectories. This evolution allows agents to leverage multi-faceted experience banks to guide strategic search frameworks like MCTS (Chen et al., 2025b) and to prevent error repetition through generalized, rule-based learning (Ouyang et al., 2025).

4.1.4 Inference-time scaling

While specialized tools and memory systems enhance specific agent capabilities, relying solely on linear execution paths often limits the exploration of complex solution spaces. To address this, inference-time scaling has emerged as a critical

paradigm to expand the search breadth and depth during problem-solving. To overcome the rigidity of sequential workflows, recent research focuses on enabling non-linear exploration via Monte Carlo Tree Search (MCTS), which facilitates flexible backtracking and qualitative feedback loops to prevent agents from stagnating in repetitive cycles (Antoniades et al., 2025). Complementing this algorithmic shift, strategies for scaling computational resources deploy multiple independent state machines in parallel, maximizing solution coverage while amortizing the high costs of context identification without the need for model re-training (Ehrlich et al., 2025). Furthermore, advanced frameworks are now integrating memory-driven scaling, utilizing time-travel mechanisms to generate diverse experiences that serve not just to resolve the immediate task, but to distill generalizable reasoning strategies for long-term agent evolution (Ouyang et al., 2025).

4.2 Training-based method

Training-based methods encompass Supervised Fine-Tuning-based (SFT-based) methods and Reinforcement Learning-based (RL) methods, utilizing resources from Section 3.1.2 to enhance the fundamental programming capabilities of LLMs.

4.2.1 SFT-based method

Supervised Fine-Tuning (SFT) serves as the primary mechanism for grounding base models in software engineering protocols. Recent efforts to achieve robust domain adaptation focus on three key dimensions: (1) *Data Scaling*. Strategies increasingly prioritize the expansion of data scale and diversity via synthesized corpora. Frameworks employ iterative generation and filtering

pipelines—augmented by automatic test generation or mid-training on billions of GitHub tokens—to comprehensively cover diverse repair scenarios (Ma et al., 2025d; Wang et al., 2025c; Yang et al., 2025e). (2) *Curriculum Learning*. Beyond raw volume, research emphasizes multi-stage curriculum learning. Models are refined through phased training sequences that progress from broad trajectory ingestion to strictly filtered, high-quality subsets or specialized tasks like localization and testing (He et al., 2025b; Rastogi et al., 2025; Liu et al., 2025a). (3) *Rejection Sampling*. To bridge the gap toward reinforcement learning, current methods employ rejection sampling pipelines. By fine-tuning exclusively on successful trajectories, these methods establish a strong baseline policy while simultaneously training verifiers to re-rank solutions at inference time (Pan et al.). See Table 3 for detailed statistics on these SFT-trained models.

4.2.2 RL-based method

Reinforcement learning optimizes issue resolution strategies through iterative interaction. This process hinges on the synergy of three core components: the algorithm for policy updates, reward design for guiding exploration, and the scaffold for managing environment rollouts. A statistical overview of recent models and their implementations across these dimensions is presented in Table 4 and Table 5.

Algorithm. The optimization of agent behaviors leverages various policy gradient and alignment strategies to stabilize learning. We discuss three common algorithmic choices as follows. (1) *Group Relative Policy Optimization (GRPO)*. A dominant approach employs GRPO, which enhances reasoning capabilities by normalizing advantages across group sampling without the heavy computational burden of a critic model (Wei et al., 2025a; Sun et al., 2025b). (2) *Proximal Policy Optimization (PPO)*. Beyond group-based methods, some approaches utilize PPO for stable updates focused on subtasks (Ma et al., 2025e). (3) *Direct Preference Optimization (DPO)*. Other work integrates MCTS with DPO to align complex, multi-step decision processes with high-quality preferred trajectories (Zhang et al., 2026).

Reward design. Effective feedback mechanisms typically incorporate both sparse outcome-based rewards and dense process-oriented signals. Most systems employ outcome reward models to pro-

vide terminal signals by utilizing strict metrics like patch similarity or detailed subtask verification ranging from localization to editing, thereby rigorously aligning outputs with ground truth (Wei et al., 2025a; Ma et al., 2025e). To mitigate the challenge of signal sparsity in long-horizon reasoning, researchers increasingly adopt process reward models and potential-based shaping techniques; these mechanisms provide dense, step-by-step feedback or token-level incentives, offering reward signals for intermediate behaviors such as context management and trajectory search throughout the reasoning process (Zeng et al., 2025b; Da et al., 2025; Sun et al., 2025b).

Scaffold. In the context of RL, a scaffold serves as the inference framework for rollouts. As statistics in Table 4 indicate, OpenHands is the most prevalent scaffold, followed by workflow-based methods (notably Agentless and two-stage workflows). Environment-native frameworks like R2E-Gym and SWE-Gym are also frequently adopted due to their seamless alignment with training data.

5 Analysis

Beyond developing new methodologies, a complementary line of research focuses on empirical analysis of existing data and methods, which provide critical insights into the limitations of current approaches and offer valuable perspectives for future research directions.

5.1 Data analysis

Recent scrutiny has exposed hidden benchmark defects, revealing that agent success rates are frequently inflated by solution leakage, ambiguous issue descriptions, and weak test suites that fail to catch incorrect patches (OpenAI, 2024; Aleithan et al., 2024; Wang et al., 2025m). Recognizing that manual cleanup is too costly and inconsistent for large-scale datasets, the field is shifting toward automating validation workflows, utilizing model-based consensus mechanisms to reliably distinguish valid fixes from false positives without human intervention (Oliva et al., 2025).

5.2 Methods analysis

Research has shifted beyond measuring simple success rates to investigate the behavioral pathology of agents. A primary focus involves diagnosing internal reasoning failures, specifically examining the tendency of models to prioritize prolonged internal

deliberation over necessary environmental interaction—a maladaptive pattern that leads to analysis paralysis and rogue actions (Cuadron et al., 2025). Complementing this, efforts to manage the complexity of massive, 128k-token interaction logs have led to streamlining trajectory inspection, utilizing novel visual interfaces to transform cryptic output streams into navigable workflows for rapid error analysis (Bula et al., 2025).

6 Application

The industrial deployment of software engineering AI has progressed from localized IDE assistance to fully autonomous systems capable of handling complex enterprise workflows. Due to space constraints, we provide a detailed discussion on these application scenarios in § A.5.

7 Challenges and Opportunities

High computational overhead. In online RL, performing concurrent rollouts necessitates the simultaneous orchestration of numerous sandboxed containers, which incurs substantial storage footprints and computational costs. Similarly, verifying instances during data construction requires extensive parallel validation. This highlights the need for lightweight sandboxing and optimized resource scheduling.

Lack of efficiency-aware evaluation. Current evaluations of issue resolution methods mainly focus on effectiveness metrics such as resolve rates while overlooking efficiency metrics like API costs and inference time. This oversight creates a biased domain where the computational and economic burdens of high-performing models are obscured. Consequently, future research must integrate both resolve rates and efficiency metrics into the evaluation framework to objectively reflect the comprehensive performance of issue resolution methods.

Limited Visually-Grounded Reasoning. Multimodal tasks are rare in current benchmarks, hindering the evaluation of visually-dependent tasks such as frontend development and data visualization. Moreover, existing methods often simply flatten visuals into text, failing to capture the critical alignment between rendering and code. To address this, future research must prioritize constructing multimodal benchmarks and training specialized code-centric models (Sun et al., 2025a).

Safety risks in autonomous resolution. Recently, some agents have exhibited unsafe behaviors on coding tasks, including deleting a user’s codebase (Business Insider, 2025) and cheating during evaluation¹. These failures motivate safer agent frameworks and more robust model safety alignment to prevent reward hacking in real deployments.

Lack of fine-grained rewards. Most RL methods for issue resolution still rely on outcome-level rewards, typically the binary test pass/fail signal. However, issue resolution requires multi-turn interaction with the environment, and an outcome reward makes credit assignment across action steps ambiguous. A promising direction is to design finer-grained process rewards to provide denser supervision and improve policy optimization.

Data leakage and contamination. As benchmarks like SWE-Bench approach saturation, evaluation reliability is threatened by significant data leakage and quality control issues. Models may inadvertently memorize solutions due to unclear training cutoff dates (Prathifkumar et al., 2025), while the benchmarks themselves frequently suffer from invalid instances—including ambiguous descriptions, solution hints, and insufficient test coverage (OpenAI, 2024; Mathews and Nagappan, 2025). To restore trust, future frameworks must prioritize rigorous data curation and decontamination protocols to guarantee the validity of comparative assessments (Zhu et al., 2025b).

Lack of autonomous context management mechanisms. Issue resolution tasks often require long-horizon, multi-turn interaction between the model and the code environment. This both raises API cost and degrades performance due to context rot (Anthropic, 2024). A promising solution is to construct an autonomous context management mechanism that proactively compresses and curates the model’s interaction history.

Insufficient patch validation and human review. Since gold tests are unavailable in real-world development, relying solely on generation capability is insufficient. Future agents should incorporate intrinsic validation mechanisms, utilizing regression testing and dependency analysis to prevent feature regression. Additionally, to bridge the trust gap,

¹<https://github.com/SWE-bench/SWE-bench/issue/s/465>

research can prioritize human-centric interfaces, such as visual explanations and concise summaries, that assist developers in efficiently reviewing and accepting model-generated solutions.

Lack of universality across SWE domains.

While existing research predominantly focuses on the implementation and integration phases of the Software Development Life Cycle (SDLC), it often fails to address the comprehensive needs of the broader software engineering field. Future research should therefore broaden its scope to encompass diverse lifecycle stages—such as requirements analysis and architectural design—to develop more versatile automated software generation methods.

8 Conclusion

This paper presents a systematic survey of LLM-based issue resolution, synthesizing 175 publicly available papers and online resources in this rapidly evolving area. We introduce a tailored taxonomy that structures prior work along three core dimensions: data, methods, and analysis. Building on this structured view, we distill key challenges and outline promising directions toward more reliable, reproducible, and practical issue-resolution systems. To facilitate continued progress, we maintain an open-source repository that tracks relevant datasets, implementations, and new developments in the field.

9 Limitations

As the first dedicated survey on issue resolution, we prioritize high-level summaries over exhaustive details due to space constraints. Our search methodology relied on citation tracking (e.g., of SWE-bench) and snowballing; while thorough, this may overlook niche or nascent works. To address this rapid evolution, we commit to continuously updating our open-source repository.

References

Pavel Adamenko, Mikhail Ivanov, Aidar Valeev, Rodion Levichev, Pavel Zadorozhny, Ivan Lopatin, Dmitry Babayev, Alena Fenogenova, and Valentin Malykh. 2025. SWE-MERA: A Dynamic Benchmark for Agenticly Evaluating Large Language Models on Software Engineering Tasks. *arXiv preprint arXiv:2507.11059*.

Sandhini Agarwal, Lama Ahmad, Jason Ai, Sam Altman, Andy Applebaum, Edwin Arbus, Rahul K Arora, Yu Bai, Bowen Baker, Haiming Bao, and 1

others. 2025. gpt-oss-120b & gpt-oss-20b model card. *arXiv preprint arXiv:2508.10925*.

Vaibhav Aggarwal, Ojasv Kamal, Abhinav Japesh, Zhi-jing Jin, and Bernhard Schölkopf. 2025. DARS: Dynamic Action Re-Sampling to Enhance Coding Agent Performance by Adaptive Tree Traversal. *arXiv preprint arXiv:2503.14269*.

Toufique Ahmed, Jatin Ganhotra, Rangeet Pan, Avraham Shinnar, Saurabh Sinha, and Martin Hirzel. 2025a. Otter: Generating tests from issues to validate SWE patches. In *Forty-second International Conference on Machine Learning*.

Toufique Ahmed, Jatin Ganhotra, Avraham Shinnar, and Martin Hirzel. 2025b. Heterogeneous Prompting and Execution Feedback for SWE Issue Test Generation and Selection. *arXiv preprint arXiv:2508.06365*.

Toufique Ahmed, Martin Hirzel, Rangeet Pan, Avraham Shinnar, and Saurabh Sinha. 2024. TDD-Bench Verified: Can LLMs Generate Tests for Issues Before They Get Resolved? *arXiv preprint arXiv:2412.02883*.

Aider-AI. 2026. Aider. <https://aider.chat/>. Accessed: 2026-01-15, from aider.chat.

Reem Aleithan. 2025. Revisiting swe-bench: On the importance of data quality for llm-based code models. In *2025 IEEE/ACM 47th International Conference on Software Engineering: Companion Proceedings (ICSE-Companion)*, pages 235–236.

Reem Aleithan, Haoran Xue, Mohammad Mahdi Mohajer, Elijah Nnorom, Gias Uddin, and Song Wang. 2024. SWE-Bench+: Enhanced Coding Benchmark for LLMs. *arXiv preprint arXiv:2410.06992*.

Anthropic. 2024. Effective Context Engineering for AI Agents. <https://www.anthropic.com/engineering/effective-context-engineering-for-ai-agents>. Anthropic Engineering Blog.

Antonis Antoniadis, Albert Örwall, Kexun Zhang, Yuxi Xie, Anirudh Goyal, and William Yang Wang. 2025. SWE-search: Enhancing software agents with monte carlo tree search and iterative refinement. In *The Thirtieth International Conference on Learning Representations*.

Ibragim Badertdinov, Alexander Golubev, Maksim Nekrashevich, Anton Shevtsov, Simon Karasik, Andrei Andriushchenko, Maria Trofimova, Daria Litvintseva, and Boris Yangel. 2025. SWE-rebench: An automated pipeline for task collection and decontaminated evaluation of software engineering agents. In *The Thirty-ninth Annual Conference on Neural Information Processing Systems Datasets and Benchmarks Track*.

Egor Bogomolov, Aleksandra Eliseeva, Timur Galimzyanov, Evgeniy Glukhov, Anton Shapkin, Maria Tigina, Yaroslav Golubev, Alexander Kovrigin, Arie van Deursen, Maliheh Izadi, and Timofey Bryksin.

2024. Long Code Arena: a Set of Benchmarks for Long-Context Code Models. *arXiv preprint arXiv:2406.11612*.
- Timothy Bula, Saurabh Pujar, Luca Buratti, Mihaela Bornea, and Avirup Sil. 2025. SeaView: Software Engineering Agent Visual Interface for Enhanced Workflow. *arXiv preprint arXiv:2504.08696*.
- Business Insider. 2025. [Replit CEO Apologizes After AI Coding Tool Deletes Company Database](#). Accessed: 2026-01-06.
- Xin Cao and Nan Yu. 2025. SIADAFIX: issue description response for adaptive program repair. *arXiv preprint arXiv:2510.16059*.
- Partha Chakraborty, Mahmoud Alfadel, and Meiyappan Nagappan. 2025. [Blaze: Cross-language and cross-project bug localization via dynamic chunking and hard example learning](#). *IEEE Transactions on Software Engineering*, 51(8):2254–2267.
- Jianming Chang, Xin Zhou, Lulu Wang, David Lo, and Bixin Li. 2025. Bridging Bug Localization and Issue Fixing: A Hierarchical Localization Framework Leveraging Large Language Models. *arXiv preprint arXiv:2502.15292*.
- Aili Chen, Aonian Li, Bangwei Gong, Binyang Jiang, Bo Fei, Bo Yang, Boji Shan, Changqing Yu, Chao Wang, Cheng Zhu, and 1 others. 2025a. Minimax-m1: Scaling test-time compute efficiently with lighting attention. *arXiv preprint arXiv:2506.13585*.
- Dong Chen, Shaoxin Lin, Muhan Zeng, Daoguang Zan, Jian-Gang Wang, Anton Cheshkov, Jun Sun, Hao Yu, Guoliang Dong, Artem Aliev, Jie Wang, Xiao Cheng, Guangtai Liang, Yuchi Ma, Pan Bian, Tao Xie, and Qianxiang Wang. 2024. CodeR: Issue Resolving with Multi-Agent and Task Graphs. *arXiv preprint arXiv:2406.01304*.
- Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde de Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, Alex Ray, Raul Puri, Gretchen Krueger, Michael Petrov, Heidy Khlaaf, Girish Sastry, Pamela Mishkin, Brooke Chan, Scott Gray, and 39 others. 2021. Evaluating Large Language Models Trained on Code. *arXiv preprint arXiv:2107.03374*.
- Silin Chen, Shaoxin Lin, Xiaodong Gu, Yuling Shi, Heng Lian, Longfei Yun, Dong Chen, Weiguo Sun, Lin Cao, and Qianxiang Wang. 2025b. SWE-Exp: Experience-Driven Software Issue Resolution. *arXiv preprint arXiv:2507.23361*.
- Yang Chen, Toufique Ahmed, Reyhaneh Jabbarvand, and Martin Hirzel. 2025c. When Old Meets New: Evaluating the Impact of Regression Tests on SWE Issue Resolution. *arXiv preprint arXiv:2510.18270*.
- Zhi Chen and Lingxiao Jiang. 2025. [Evaluating software development agents: Patch patterns, code quality, and issue complexity in real-world github scenarios](#). In *2025 IEEE International Conference on Software Analysis, Evolution and Reengineering (SANER)*, page 657–668. IEEE.
- Zhi Chen, Wei Ma, and Lingxiao Jiang. 2025d. Beyond Final Code: A Process-Oriented Error Analysis of Software Development Agents in Real-World GitHub Scenarios. *arXiv preprint arXiv:2503.12374*.
- Zhilong Chen, Chengzong Zhao, Boyuan Chen, Dayi Lin, Yihao Chen, Arthur Leung, Gopi Krishnan Rajbahadur, Gustavo A. Oliva, Haoxiang Zhang, Aaditya Bhatia, Chong Chun Yong, and Ahmed E. Hassan. 2025e. RepoForge: Training a SOTA Fast-thinking SWE Agent with an End-to-End Data Curation Pipeline Synergizing SFT and RL at Scale. *arXiv preprint arXiv:2508.01550*.
- Zimin Chen, Yue Pan, Siyu Lu, Jiayi Xu, Claire Le Goues, Martin Monperrus, and He Ye. 2025f. Prometheus: Unified Knowledge Graphs for Issue Resolution in Multilingual Codebases. *arXiv preprint arXiv:2507.19942*.
- Anton Cheshkov, Pavel Zadorozhny, Rodion Levichev, Evgeny Maslov, and Ronaldo Franco Jaldin. 2024. Exploring the Potential of Conversational Test Suite Based Program Repair on SWE-bench. *arXiv preprint arXiv:2410.04485*.
- Cognition-Team. 2025. SWE-bench technical report. <https://cognition.ai/blog/swe-bench-technical-report>. Accessed: 2026-01-15, from Cognition.ai.
- Alejandro Cuadron, Dacheng Li, Wenjie Ma, Xingyao Wang, Yichuan Wang, Siyuan Zhuang, Shu Liu, Luis Gaspar Schroeder, Tian Xia, Huanzhi Mao, Nicholas Thumiger, Aditya Desai, Ion Stoica, Ana Klimovic, Graham Neubig, and Joseph E. Gonzalez. 2025. The Danger of Overthinking: Examining the Reasoning-Action Dilemma in Agentic Tasks. *arXiv preprint arXiv:2502.08235*.
- Jeff Da, Clinton Wang, Xiang Deng, Yuntao Ma, Nikhil Barhate, and Sean Hendryx. 2025. Agent-RLVR: Training Software Engineering Agents via Guidance and Environment Rewards. *arXiv preprint arXiv:2506.11425*.
- Hankun Dai, Maoquan Wang, Mengnan Qi, Yikai Zhang, Zijian Jin, Yongqiang Yao, Yufan Huang, Shengyu Fu, and Elsie Nallipogu. 2025. Lita: Light Agent Uncovers the Agentic Coding Capabilities of LLMs. *arXiv preprint arXiv:2509.25873*.
- DeepSeek-AI, Aixin Liu, Aoxue Mei, Bangcai Lin, Bing Xue, Bingxuan Wang, Bingzheng Xu, Bochao Wu, Bowei Zhang, Chaofan Lin, Chen Dong, Chengda Lu, Chenggang Zhao, Chengqi Deng, Chenhao Xu, Chong Ruan, Damai Dai, Daya Guo, Dejian Yang, and 245 others. 2025. DeepSeek-V3.2: Pushing the Frontier of Open Large Language Models. *arXiv preprint arXiv:2512.02556*.
- Xiang Deng, Jeff Da, Edwin Pan, Yannis Yiming He, Charles Ide, Kanak Garg, Niklas Lauffer, Andrew

- Park, Nitin Pasari, Chetan Rane, Karmini Sampath, Maya Krishnan, Srivatsa Kundurthy, Sean Hendryx, Zifan Wang, Vijay Bharadwaj, Jeff Holm, Raja Aluri, Chen Bo Calvin Zhang, and 3 others. 2025a. SWE-Bench Pro: Can AI Agents Solve Long-Horizon Software Engineering Tasks? *arXiv preprint arXiv:2509.16941*.
- Yihe Deng, I-Hung Hsu, Jun Yan, Zifeng Wang, Ru-jun Han, Gufeng Zhang, Yanfei Chen, Wei Wang, Tomas Pfister, and Chen-Yu Lee. 2025b. Supervised Reinforcement Learning: From Expert Trajectories to Step-wise Reasoning. *arXiv preprint arXiv:2510.25992*.
- Devlo. 2026. Achieving SOTA on SWE-bench. <http://devlo.ai/blog/devlo-swe-bench-sota/>. Accessed: 2026-01-15, from devlo.
- Yaxin Du, Yuzhu Cai, Yifan Zhou, Cheng Wang, Yu Qian, Xianghe Pang, Qian Liu, Yue Hu, and Siheng Chen. 2025. SWE-Dev: Evaluating and Training Autonomous Feature-Driven Software Development. *arXiv preprint arXiv:2505.16975*.
- Ryan Ehrlich, Bradley Brown, Jordan Juravsky, Ronald Clark, Christopher Ré, and Azalia Mirhoseini. 2025. CodeMonkeys: Scaling Test-Time Compute for Software Engineering. *arXiv preprint arXiv:2501.14723*.
- Aleksandra Eliseeva, Alexander Kovrigin, Ilia Kholkin, Egor Bogomolov, and Yaroslav Zharov. 2025. [Envbench: A benchmark for automated environment setup](#). In *ICLR 2025 Third Workshop on Deep Learning for Code*.
- Kelin Fu, Tianyu Liu, Zeyu Shang, Yingwei Ma, Jian Yang, Jiaheng Liu, and Kaigui Bian. 2025. Multi-Docker-Eval: A ‘Shovel of the Gold Rush’ Benchmark on Automatic Environment Building for Software Engineering. *arXiv preprint arXiv:2512.06915*.
- Shubham Gandhi, Atharva Naik, Yiqing Xie, and Carolyn Rose. 2025a. An Empirical Study on Strong-Weak Model Collaboration for Repo-level Code Generation. *arXiv preprint arXiv:2505.20182*.
- Shubham Gandhi, Jason Tsay, Jatin Ganhotra, Kiran Kate, and Yara Rizk. 2025b. When Agents go Astray: Course-Correcting SWE Agents with PRMs. *arXiv preprint arXiv:2509.02360*.
- Spandan Garg, Benjamin Steenhoeck, and Yufan Huang. 2025. Saving SWE-Bench: A Benchmark Mutation Approach for Realistic Agent Evaluation. *arXiv preprint arXiv:2510.08996*.
- Anmol Gautam, Kishore Kumar, Adarsh Jha, Mukunda NS, and Ishaan Bhola. 2024. SuperCoder2.0: Technical Report on Exploring the feasibility of LLMs as Autonomous Programmer. *arXiv preprint arXiv:2409.11190*.
- Alexander Golubev, Maria Trofimova, Sergei Polezhaev, Ibragim Badertdinov, Maksim Nekrashevich, Anton Shevtsov, Simon Karasik, Sergey Abramov, Andrei Andriushchenko, Filipp Fisin, Sergei Skvortsov, and Boris Yangel. 2025. Training Long-Context, Multi-Turn Software Engineering Agents with Reinforcement Learning. *arXiv preprint arXiv:2508.03501*.
- Jiale Guo, Suizhi Huang, Mei Li, Dong Huang, Xingsheng Chen, Regina Zhang, Zhijiang Guo, Han Yu, Siu-Ming Yiu, Pietro Lio, and Kwok-Yan Lam. 2025a. A Comprehensive Survey on Benchmarks and Solutions in Software Engineering of LLM-Empowered Agentic System. *arXiv preprint arXiv:2510.09721*.
- Lianghong Guo, Wei Tao, Runhan Jiang, Yanlin Wang, Jiachi Chen, Xilin Liu, Yuchi Ma, Mingzhi Mao, Hongyu Zhang, and Zibin Zheng. 2025b. Omnigirl: A multilingual and multimodal benchmark for github issue resolution. *Proceedings of the ACM on Software Engineering*, 2(ISSTA):24–46.
- Lianghong Guo, Yanlin Wang, Caihua Li, Wei Tao, Pengyu Yang, Jiachi Chen, Haoyu Song, Duyu Tang, and Zibin Zheng. 2025c. SWE-Factory: Your Automated Factory for Issue Resolution Training Data and Evaluation Benchmarks. *arXiv preprint arXiv:2506.10954*.
- Yifu Guo, Jiaye Lin, Huacan Wang, Yuzhen Han, Sen Hu, Ziyi Ni, Licheng Wang, and Mingguang Chen. 2025d. [SE-agent: Self-evolution trajectory optimization in multi-step reasoning with LLM-based agents](#). In *The Thirty-ninth Annual Conference on Neural Information Processing Systems*.
- Dhruv Gupta, Gayathri Ganesh Lakshmy, and Yiqing Xie. 2025. SACL: Understanding and Combating Textual Bias in Code Retrieval with Semantic-Augmented Reranking and Localization. *arXiv preprint arXiv:2506.20081*.
- Kevin Han, Siddharth Maddikayala, Tim Knappe, Om Patel, Austen Liao, and Amir Barati Farmani. 2025. TDFlow: Agentic Workflows for Test Driven Software Engineering. *arXiv preprint arXiv:2510.23761*.
- Xinyi He, Qian Liu, Mingzhe Du, Lin Yan, ZhiJie Fan, Yiming Huang, Zejian Yuan, and Zejun MA. 2025a. [SWE-perf: Can language models optimize code performance on real-world repositories?](#) In *NeurIPS 2025 Fourth Workshop on Deep Learning for Code*.
- Zhenyu He, Qingping Yang, Wei Sheng, Xiaojian Zhong, Kechi Zhang, Chenxin An, Wenlei Shi, Tianle Cai, Di He, Jiaze Chen, and Jingjing Xu. 2025b. Swiss: A multi-task fine-tuning and rl recipe for high-performance issue resolution. <https://www.notion.so/SWE-Swiss-A-Multi-Task-Fine-Tuning-and-RL-Recipe-for-High-Performance-Issue-Resolution-21e174dedd4880ea829ed4c861c44f88>. Notion Blog.
- Sirui Hong, Mingchen Zhuge, Jonathan Chen, Xiawu Zheng, Yuheng Cheng, Jinlin Wang, Ceyao Zhang, Zili Wang, Steven Ka Shing Yau, Zijuan Lin, Liyang Zhou, Chenyu Ran, Lingfeng Xiao, Chenglin Wu,

- and Jürgen Schmidhuber. 2024. Metagpt: Meta programming for A multi-agent collaborative framework. In *ICLR*. OpenReview.net.
- Hao Hu, Hongyu Zhang, Jifeng Xuan, and Weigang Sun. 2014. [Effective bug triage based on historical bug-fix information](#). In *Proceedings of the 2014 IEEE 25th International Symposium on Software Reliability Engineering*, ISSRE '14, page 122–132, USA. IEEE Computer Society.
- Kai Huang, Zhengzi Xu, Su Yang, Hongyu Sun, Xuejun Li, Zheng Yan, and Yuqing Zhang. 2023. A Survey on Automated Program Repair Techniques. *arXiv preprint arXiv:2303.18184*.
- Kai Huang, Jian Zhang, Xiaofei Xie, and Chunyang Chen. 2025. Seeing is Fixing: Cross-modal reasoning with multimodal LLMs for visual software issue fixing. In *ASE*. IEEE/ACM.
- Naman Jain, Jaskirat Singh, Manish Shetty, Tianjun Zhang, Liang Zheng, Koushik Sen, and Ion Stoica. 2025. [R2e-gym: Procedural environment generation and hybrid verifiers for scaling open-weights SWE agents](#). In *Second Conference on Language Modeling*.
- Juyong Jiang, Fan Wang, Jiasi Shen, Sungju Kim, and Sunghun Kim. 2025a. [A survey on large language models for code generation](#). *ACM Trans. Softw. Eng. Methodol.*
- Mingjian Jiang, Yangjun Ruan, Luis Lastras, Pavan Kanipathi, and Tatsunori Hashimoto. 2025b. Putting It All into Context: Simplifying Agents with LCLMs. *arXiv preprint arXiv:2505.08120*.
- Zhonghao Jiang, Xiaoxue Ren, Meng Yan, Wei Jiang, Yong Li, and Zhongxin Liu. 2025c. Issue localization via LLM-driven iterative code graph searching. In *ASE*. IEEE/ACM.
- Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik R. Narasimhan. 2024. Swe-bench: Can language models resolve real-world github issues? In *ICLR*. OpenReview.net.
- Lara Khatib, Noble Saji Mathews, and Meiyappan Nagappan. 2025. AssertFlip: Reproducing Bugs via Inversion of LLM-Generated Passing Tests. *arXiv preprint arXiv:2507.17542*.
- Konstantinos Kitsios, Marco Castelluccio, and Alberto Bacchelli. 2025. Automated Generation of Issue-Reproducing Tests by Combining LLMs and Search-Based Testing. *arXiv preprint arXiv:2509.01616*.
- Alexander Kovrigin, Aleksandra Eliseeva, Konstantin Grotov, Egor Bogomolov, and Yaroslav Zharov. 2025. PIPer: On-Device Environment Setup via Online Reinforcement Learning. *arXiv preprint arXiv:2509.25455*.
- Alexander Kovrigin, Aleksandra Eliseeva, Yaroslav Zharov, and Timofey Bryksin. 2024. On The Importance of Reasoning for Context Retrieval in Repository-Level Code Editing. *arXiv preprint arXiv:2406.04464*.
- Bin Lei, Weitai Kang, Zijian Zhang, Winson Chen, Xi Xie, Shan Zuo, Mimi Xie, Ali Payani, Mingyi Hong, Yan Yan, and Caiwen Ding. 2025. InfantAgent-Next: A Multimodal Generalist Agent for Automated Computer Interaction. *arXiv preprint arXiv:2505.10887*.
- Bin Lei, Yuchen Li, Yiming Zeng, Tao Ren, Yi Luo, Tianyu Shi, Zitian Gao, Zeyu Hu, Weitai Kang, and Qiuwu Chen. 2024. Infant Agent: A Tool-Integrated, Logic-Driven Agent with Cost-Effective API Usage. *arXiv preprint arXiv:2411.01114*.
- Han Li, Yuling Shi, Shaoxin Lin, Xiaodong Gu, Heng Lian, Xin Wang, Yantao Jia, Tao Huang, and Qianxiang Wang. 2026. SWE-Debate: Competitive multi-agent debate for software issue resolution. In *ICSE*. ACM.
- Hongwei Li, Yuheng Tang, Shiqi Wang, and Wenbo Guo. 2025a. [Patchpilot: A cost-efficient software engineering agent with early attempts on formal verification](#). In *Forty-second International Conference on Machine Learning*.
- KeFan Li, Mengfei Wang, Hengzhi Zhang, Zhichao Li, Yuan Yuan, Mu Li, Xiang Gao, Hailong Sun, Chunming Hu, and Weifeng Lv. 2025b. InfCode: Adversarial Iterative Refinement of Tests and Patches for Reliable Software Issue Resolution. *arXiv preprint arXiv:2511.16004*.
- Wei Li, Xin Zhang, Zhongxin Guo, Shaoguang Mao, Wen Luo, Guangyue Peng, Yangyu Huang, Houfeng Wang, and Scarlett Li. 2025c. [Fea-bench: A benchmark for evaluating repository-level code generation for feature implementation](#). In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, ACL 2025, Vienna, Austria, July 27 - August 1, 2025, pages 17160–17176. Association for Computational Linguistics.
- Shanchao Liang, Spandan Garg, and Roshanak Zilouchian Moghaddam. 2025. The SWE-Bench Illusion: When State-of-the-Art LLMs Remember Instead of Reason. *arXiv preprint arXiv:2506.12286*.
- Yalan Lin, Yingwei Ma, Rongyu Cao, Binhua Li, Fei Huang, Xiaodong Gu, and Yongbin Li. 2024. LLMs as Continuous Learners: Improving the Reproduction of Defective Code in Software Issues. *arXiv preprint arXiv:2411.13941*.
- Bingchang Liu, Chaoyu Chen, Zi Gong, Cong Liao, Huan Wang, Zhichao Lei, Ming Liang, Dajun Chen, Min Shen, Hailian Zhou, Wei Jiang, Hang Yu, and Jianguo Li. 2024a. [Mftcoder: Boosting code llms with multitask fine-tuning](#). In *Proceedings of the*

- 30th ACM SIGKDD Conference on Knowledge Discovery and Data Mining, KDD '24, page 5430–5441, New York, NY, USA. Association for Computing Machinery.
- Shukai Liu, Jian Yang, Bo Jiang, Yizhi Li, Jinyang Guo, Xianglong Liu, and Bryan Dai. 2025a. Context as a Tool: Context Management for Long-Horizon SWE-Agents. *arXiv preprint arXiv:2512.22087*.
- Simiao Liu, Fang Liu, Liehao Li, Xin Tan, Yinghao Zhu, Xiaoli Lian, and Li Zhang. 2025b. An Empirical Study on Failures in Automated Issue Solving. *arXiv preprint arXiv:2509.13941*.
- Wei Liu, Chao Peng, Pengfei Gao, Aofan Liu, Wei Zhang, Haiyan Zhao, and Zhi Jin. 2025c. GraphLocator: Graph-guided Causal Reasoning for Issue Localization. *arXiv preprint arXiv:2512.22469*.
- Ye Liu, Rui Meng, Shafiq Joty, silvio savarese, Caiming Xiong, Yingbo Zhou, and Semih Yavuz. 2025d. CodeXEmbed: A generalist embedding model family for multilingual and multi-task code retrieval. In *Second Conference on Language Modeling*.
- Yizhou Liu, Pengfei Gao, Xinchun Wang, Jie Liu, Yexuan Shi, Zhao Zhang, and Chao Peng. 2024b. MarsCode Agent: AI-native Automated Bug Fixing. *arXiv preprint arXiv:2409.00899*.
- LogicStar-AI and SRILab. 2025. Agents in the Wild. <https://insights.logicstar.ai/>. Accessed: 2026-01-15, from logicstar.ai.
- Jane Luo, Xin Zhang, Steven Liu, Jie Wu, Jianfeng Liu, Yiming Huang, Yangyu Huang, Chengyu Yin, Ying Xin, Yuefeng Zhan, Hao Sun, Qi Chen, Scarlett Li, and Mao Yang. 2025a. RPG: A Repository Planning Graph for Unified and Scalable Codebase Generation. *arXiv preprint arXiv:2509.16198*.
- Michael Luo, Naman Jain, Jaskirat Singh, Sijun Tan, Ameen Patel, Qingyang Wu, Alpay Ariyak, Colin Cai, Tarun Venkat, Shang Zhu, Ben Athiwaratkun, Manan Roongta, Ce Zhang, Li Erran Li, Raluca Ada Popa, Koushik Sen, and Ion Stoica. 2025b. DeepSWE: Training a state-of-the-art coding agent from scratch by scaling rl. <https://pretty-radio-b75.notion.site/DeepSWE-Training-a-Fully-Open-sourced-State-of-the-Art-Coding-Agent-by-Scaling-RL-22281902c1468193aabb9a8c59bbe33>. Notion Blog.
- Jeffrey Jian Ma, Milad Hashemi, Amir Yazdanbakhsh, Kevin Swersky, Ofir Press, Enhui Li, Vijay Janapa Reddi, and Parthasarathy Ranganathan. 2025a. SWEfficiency: Can Language Models Optimize Real-World Repositories on Real Workloads? *arXiv preprint arXiv:2511.06090*.
- Yingwei Ma, Rongyu Cao, Yongchang Cao, Yue Zhang, Jue Chen, Yibo Liu, Yuchen Liu, Binhua Li, Fei Huang, and Yongbin Li. 2025b. Swe-gpt: A process-centric language model for automated software improvement. *Proceedings of the ACM on Software Engineering*, 2(ISSTA):2362–2383.
- Yingwei Ma, Yongbin Li, Yihong Dong, Xue Jiang, Rongyu Cao, Jue Chen, Fei Huang, and Binhua Li. 2025c. Thinking Longer, Not Larger: Enhancing Software Engineering Agents via Scaling Test-Time Compute. *arXiv preprint arXiv:2503.23803*.
- Yingwei Ma, Qingping Yang, Rongyu Cao, Binhua Li, Fei Huang, and Yongbin Li. 2025d. Alibaba lingmaagent: Improving automated issue resolution via comprehensive repository exploration. In *Proceedings of the 33rd ACM International Conference on the Foundations of Software Engineering, FSE Companion '25*, page 238–249. ACM.
- Zexiong Ma, Shengnan An, Zeqi Lin, Yanzhen Zou, and Bing Xie. 2024. Repository Structure-Aware Training Makes SLMs Better Issue Resolver. *arXiv preprint arXiv:2412.19031*.
- Zexiong Ma, Chao Peng, Pengfei Gao, Xiangxin Meng, Yanzhen Zou, and Bing Xie. 2025e. Sorft: Issue resolving with subtask-oriented reinforced fine-tuning. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers), ACL 2025, Vienna, Austria, July 27 - August 1, 2025*, pages 11427–11441. Association for Computational Linguistics.
- Zexiong Ma, Chao Peng, Qunhong Zeng, Pengfei Gao, Yanzhen Zou, and Bing Xie. 2025f. Tool-integrated Reinforcement Learning for Repo Deep Search. *arXiv preprint arXiv:2508.03012*.
- Matias Martinez and Xavier Franch. 2025. Dissecting the SWE-Bench Leaderboards: Profiling Submitters and Architectures of LLM- and Agent-Based Repair Systems. *arXiv preprint arXiv:2506.17208*.
- Noble Saji Mathews and Meiyappan Nagappan. 2025. Is Your Automated Software Engineer Trustworthy? *arXiv preprint arXiv:2506.17812*.
- Meituan-LongCat-Team, Anchun Gui, Bei Li, Bingyang Tao, Bole Zhou, Borun Chen, Chao Zhang, Chao Zhang, Chengcheng Han, Chenhui Yang, Chi Zhang, Chong Peng, Chuyu Zhang, Cong Chen, Fengcun Li, Gang Xu, Guoyuan Lin, Hao Jiang, Hao Liang, and 108 others. 2025. Introducing LongCat-Flash-Thinking: A Technical Report. *arXiv preprint arXiv:2509.18883*.
- Sanket Mhatre, Yasharth Bajpai, Sumit Gulwani, Emerson Murphy-Hill, and Gustavo Soares. 2025. SWE-Sharp: A Reproducible Benchmark for C# Software Engineering Tasks. *arXiv preprint arXiv:2511.02352*.
- Samuel Miserendino, Michele Wang, Tejal Patwardhan, and Johannes Heidecke. 2025. Swe-lancer: Can frontier llms earn \$1 million from real-world freelance software engineering? In *ICML*. OpenReview.net.
- Fangwen Mu, Junjie Wang, Lin Shi, Song Wang, Shoubin Li, and Qing Wang. 2025. EXPEREPAIR: Dual-Memory Enhanced LLM-based Repository-Level Program Repair. *arXiv preprint arXiv:2506.10484*.

- Niels Mündler, Mark Niklas Müller, Jingxuan He, and Martin Vechev. 2024. [Swt-bench: Testing and validating real-world bug-fixes with code agents](#). In *Advances in Neural Information Processing Systems*, volume 37, pages 81857–81887. Curran Associates, Inc.
- Noor Nashid, Islem Bouzenia, Michael Pradel, and Ali Mesbah. 2025. Issue2Test: Generating Reproducing Test Cases from Issue Reports. *arXiv preprint arXiv:2503.16320*.
- Nebius. 2024. Scaling data collection for training SWE agents. <https://nebius.com/blog/posts/scaling-data-collection-for-training-swe-agents>. Accessed: 2025-12-12.
- Gustavo A. Oliva, Gopi Krishnan Rajbahadur, Aaditya Bhatia, Haoxiang Zhang, Yihao Chen, Zhilong Chen, Arthur Leung, Dayi Lin, Boyuan Chen, and Ahmed E. Hassan. 2025. SPICE: An automated SWE-Bench labeling pipeline for issue clarity, test coverage, and effort estimation. In *ASE*. IEEE/ACM.
- OpenAI. 2024. [Introducing swe-bench verified | openai](#). [Online; accessed 2025-09-22].
- Siru Ouyang, Jun Yan, I-Hung Hsu, Yanfei Chen, Ke Jiang, Zifeng Wang, Rujun Han, Long T. Le, Samira Daruki, Xiangru Tang, Vishy Tirumalashetty, George Lee, Mahsan Rofouei, Hangfei Lin, Jiawei Han, Chen-Yu Lee, and Tomas Pfister. 2025. ReasoningBank: Scaling Agent Self-Evolving with Reasoning Memory. *arXiv preprint arXiv:2509.25140*.
- Siru Ouyang, Wenhao Yu, Kaixin Ma, Zilin Xiao, Zhihan Zhang, Mengzhao Jia, Jiawei Han, Hongming Zhang, and Dong Yu. Repograph: Enhancing ai software engineering with repository-level code graph. In *The Thirteenth International Conference on Learning Representations*.
- Anvith Pabba, Alex Mathai, Anindya Chakraborty, and Baishakhi Ray. 2025. SemAgent: A Semantics Aware Program Repair Agent. *arXiv preprint arXiv:2506.16650*.
- Jiayi Pan, Xingyao Wang, Graham Neubig, Navdeep Jaitly, Heng Ji, Alane Suhr, and Yizhe Zhang. Training software engineering agents and verifiers with swe-gym. In *Forty-second International Conference on Machine Learning*.
- Ruwei Pan, Hongyu Zhang, and Chao Liu. 2025. CodeCoR: An LLM-Based Self-Reflective Multi-Agent Framework for Code Generation. *arXiv preprint arXiv:2501.07811*.
- Minh V. T. Pham, Huy N. Phan, Hoang N. Phan, Cuong Le Chi, Tien N. Nguyen, and Nghi D. Q. Bui. 2025. SWE-Synth: Synthesizing Verifiable Bug-Fix Data to Enable Large Language Models in Resolving Real-World Bugs. *arXiv preprint arXiv:2504.14757*.
- Huy Nhat Phan, Tien N. Nguyen, Phong X. Nguyen, and Nghi D. Q. Bui. 2024. HyperAgent: Generalist Software Engineering Agents to Solve Coding Tasks at Scale. *arXiv preprint arXiv:2409.16299*.
- Thanosan Prathifkumar, Noble Saji Mathews, and Meiyappan Nagappan. 2025. Does SWE-Bench-Verified Test Agent Ability or Model Memory? *arXiv preprint arXiv:2512.10218*.
- Binhang Qi, Hailong Sun, Wei Yuan, Hongyu Zhang, and Xiangxin Meng. 2022. [Dreamloc: A deep relevance matching-based framework for bug localization](#). *IEEE Transactions on Reliability*, 71(1):235–249.
- Chen Qian, Wei Liu, Hongzhang Liu, Nuo Chen, Yufan Dang, Jiahao Li, Cheng Yang, Weize Chen, Yusheng Su, Xin Cong, Juyuan Xu, Dahai Li, Zhiyuan Liu, and Maosong Sun. 2024. [Chatdev: Communicative agents for software development](#). In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, page 15174–15186. Association for Computational Linguistics.
- Quadropic-Team. 2025. Quadropic Insiders : Syntheo Tops Swelite Feb. <https://insiders.quadropic.com/insiders/syntheo-tops-swelite-feb>. Accessed: 2026-01-15, from insiders.quadropic.com.
- Mohit Raghavendra, Anisha Gunjal, Bing Liu, and Yunzhong He. 2026. Agentic Rubrics as Contextual Verifiers for SWE Agents. *arXiv preprint arXiv:2601.04171*.
- Muhammad Shihab Rashid, Christian Bock, Yuan Zhuang, Alexander Buchholz, Tim Esler, Simon Valentin, Luca Franceschi, Martin Wistuba, Prabhu Teja Sivaprasad, Woo Jung Kim, Anoop Deoras, Giovanni Zappella, and Laurent Callot. 2025. SWE-PolyBench: A multi-language benchmark for repository level evaluation of coding agents. *arXiv preprint arXiv:2504.08703*.
- Abhinav Rastogi, Adam Yang, Albert Q. Jiang, Alexander H. Liu, Alexandre Sablayrolles, Amélie Héliou, Amélie Martin, Anmol Agarwal, Andy Ehrenberg, Andy Lo, Antoine Roux, Arthur Darcet, Arthur Mensch, Baptiste Bout, Baptiste Rozière, Baudouin De Monicault, Chris Bamford, Christian Wallenwein, Christophe Renaudin, and 84 others. 2025. Devstral: Fine-tuning Language Models for Coding Agent Applications. *arXiv preprint arXiv:2509.25193*.
- Revanth Gangi Reddy, Tarun Suresh, JaeHyeok Doo, Ye Liu, Xuan Phi Nguyen, Yingbo Zhou, Semih Yavuz, Caiming Xiong, Heng Ji, and Shafiq Joty. 2025. SweRank: Software Issue Localization with Code Ranking. *arXiv preprint arXiv:2505.07849*.
- Haifeng Ruan, Yuntong Zhang, and Abhik Roychoudhury. 2025. Specrover: Code intent extraction via llms. In *ICSE*, pages 963–974. IEEE.

- Amirali Sajadi, Kostadin Damevski, and Preetha Chatterjee. 2025. How Safe Are AI-Generated Patches? A Large-scale Study on Security Risks in LLM and Agentic Automated Program Repair on SWE-bench. *arXiv preprint arXiv:2507.02976*.
- ByteDance Seed, :, Jiase Chen, Tiantian Fan, Xin Liu, Lingjun Liu, Zhiqi Lin, Mingxuan Wang, Chengyi Wang, Xiangpeng Wei, Wenyuan Xu, Yufeng Yuan, Yu Yue, Lin Yan, Qiyang Yu, Xiaochen Zuo, Chi Zhang, Ruofei Zhu, Zhecheng An, and 255 others. 2025. Seed1.5-Thinking: Advancing Superb Reasoning Models with Reinforcement Learning. *arXiv preprint arXiv:2504.13914*.
- Manish Shetty, Naman Jain, Jinjian Liu, Vijay Kethanaboyina, Koushik Sen, and Ion Stoica. 2025. GSO: Challenging Software Optimization Tasks for Evaluating SWE-Agents. *arXiv preprint arXiv:2505.23671*.
- KaShun Shum, Binyuan Hui, Jiawei Chen, Lei Zhang, X. W., Jiayi Yang, Yuzhen Huang, Junyang Lin, and Junxian He. 2025. SWE-RM: Execution-free Feedback For Software Engineering Agents. *arXiv preprint arXiv:2512.21919*.
- Atefeh Sohrabizadeh, Jialin Song, Mingjie Liu, Rajarshi Roy, Chankyu Lee, Jonathan Raiman, and Bryan Catanzaro. 2025. [Nemotron-CORTEXA: Enhancing LLM agents for software engineering tasks via improved localization and solution diversity](#). In *Forty-second International Conference on Machine Learning*.
- Aditya Bharat Soni, Boxuan Li, Xingyao Wang, Valerie Chen, and Graham Neubig. Coding agents with multimodal browsing are generalist problem solvers. In *ICML 2025 Workshop on Computer Use Agents*.
- Atharv Sonwane, Isadora White, Hyunji Lee, Matheus Pereira, Lucas Caccia, Minseon Kim, Zhengyan Shi, Chinmay Singh, Alessandro Sordani, Marc-Alexandre Côté, and Xingdi Yuan. 2025. BugPilot: Complex Bug Generation for Efficient Learning of SWE Skills. *arXiv preprint arXiv:2510.19898*.
- Hongjin SU, Ruoxi Sun, Jinsung Yoon, Pengcheng Yin, Tao Yu, and Serkan O Arik. 2025. [Learn-by-interact: A data-centric framework for self-adaptive agents in realistic environments](#). In *The Thirteenth International Conference on Learning Representations*.
- Qiushi Sun, Jingyang Gong, Yang Liu, Qiaosheng Chen, Lei Li, Kai Chen, Qipeng Guo, Ben Kao, and Fei Yuan. 2025a. JanusCoder: Towards a Foundational Visual-Programmatic Interface for Code Intelligence. *arXiv preprint arXiv:2510.23538*.
- Weiwei Sun, Miao Lu, Zhan Ling, Kang Liu, Xuesong Yao, Yiming Yang, and Jiecao Chen. 2025b. Scaling Long-Horizon LLM Agent via Context-Folding. *arXiv preprint arXiv:2510.11967*.
- Tarun Suresh, Revanth Gangi Reddy, Yifei Xu, Zach Nussbaum, Andriy Mulyar, Brandon Duderstadt, and Heng Ji. 2025. [CoRNStack: High-quality contrastive data for better code retrieval and reranking](#). In *The Thirteenth International Conference on Learning Representations*.
- Xiangru Tang, Tianrui Qin, Tianhao Peng, Ziyang Zhou, Daniel Shao, Tingting Du, Xinming Wei, Peng Xia, Fang Wu, He Zhu, Ge Zhang, Jiaheng Liu, Xingyao Wang, Sirui Hong, Chenglin Wu, Hao Cheng, Chi Wang, and Wangchunshu Zhou. 2025a. Agent KB: Leveraging Cross-Domain Experience for Agentic Problem Solving. *arXiv preprint arXiv:2507.06229*.
- Xunzhu Tang, Jiechao Gao, Jin Xu, Tiezhu Sun, Yewei Song, Saad Ezzini, Wendkûni C. Ouédraogo, Jacques Klein, and Tegawendé F. Bissyandé. 2025b. [SynFix: Dependency-aware program repair via RelationGraph analysis](#). In *Findings of the Association for Computational Linguistics: ACL 2025*, pages 4878–4894, Vienna, Austria. Association for Computational Linguistics.
- Yuheng Tang, Hongwei Li, Kaijie Zhu, Michael Yang, Yangruibo Ding, and Wenbo Guo. 2025c. Co-PatcheR: Collaborative Software Patching with Component(s)-specific Small Reasoning Models. *arXiv preprint arXiv:2505.18955*.
- Chaofan Tao, Jierun Chen, Yuxin Jiang, Kaiqi Kou, Shaowei Wang, Ruoyu Wang, Xiaohui Li, Sidi Yang, Yiming Du, Jianbo Dai, Zhiming Mao, Xinyu Wang, Lifeng Shang, and Haoli Bai. 2026. SWE-Lego: Pushing the Limits of Supervised Fine-tuning for Software Issue Resolving. *arXiv preprint arXiv:2601.01426*.
- Wei Tao, Yucheng Zhou, Yanlin Wang, Wenqiang Zhang, Hongyu Zhang, and Yu Cheng. 2024. [MAGIS: LLM-based multi-agent framework for github issue resolution](#). In *The Thirty-eighth Annual Conference on Neural Information Processing Systems*.
- Natalia Tarasova, Enrique Balp-Straffon, Aleksei Iancheruk, Yevhenii Sielskyi, Nikita Kozodoi, Liam H. Byrne, Jack Butler, Dayuan jiang, Marcin Czelej, Andrew Ang, Yash Shah, Roi Blanco, and Sergei Ivanov. 2025. [SWE-infrabench: Evaluating language models on cloud infrastructure code](#). In *NeurIPS 2025 Workshop on Evaluating the Evolving LLM Lifecycle: Benchmarks, Emergent Abilities, and Scaling*.
- Vali Tawosi, Salwa Alamir, Xiaomo Liu, and Manuela Veloso. 2025. Meta-RAG on Large Codebases Using Code Summarization. *arXiv preprint arXiv:2508.02611*.
- FAIR CodeGen team, Jade Copet, Quentin Carbonneaux, Gal Cohen, Jonas Gehring, Jacob Kahn, Jan-nik Kossen, Felix Kreuk, Emily McMilin, Michel Meyer, Yuxiang Wei, David Zhang, Kunhao Zheng, Jordi Armengol-Estapé, Pedram Bashiri, Maximilian Beck, Pierre Chambon, Abhishek Charnalia, Chris

- Cummins, and 32 others. 2025. CWM: An Open-Weights LLM for Research on Code Generation with World Models. *arXiv preprint arXiv:2510.02387*.
- Kimi Team, Yifan Bai, Yiping Bao, Guanduo Chen, Jiahao Chen, Ningxin Chen, Ruijue Chen, Yanru Chen, Yuankun Chen, Yutian Chen, Zhuofu Chen, Jialei Cui, Hao Ding, Mengnan Dong, Angang Du, Chenzhuang Du, Dikang Du, Yulun Du, Yu Fan, and 150 others. 2025a. Kimi K2: Open Agentic Intelligence. *arXiv preprint arXiv:2507.20534*.
- Trae Research Team, Pengfei Gao, Zhao Tian, Xiangxin Meng, Xincheng Wang, Ruida Hu, Yuanan Xiao, Yizhou Liu, Zhao Zhang, Junjie Chen, Cuiyun Gao, Yun Lin, Yingfei Xiong, Chao Peng, and Xia Liu. 2025b. Trae Agent: An LLM-based Agent for Software Engineering with Test-time Scaling. *arXiv preprint arXiv:2507.23370*.
- Minh V. T. Thai, Tue Le, Dung Nguyen Manh, Huy Phan Nhat, and Nghi D. Q. Bui. 2025. SWE-EVO: Benchmarking Coding Agents in Long-Horizon Software Evolution Scenarios. *arXiv preprint arXiv:2512.18470*.
- Arihant Tripathy, Ch Pavan Harshit, and Karthik Vaidhyathan. 2026. SWEnergy: An empirical study on energy efficiency in agentic issue resolution frameworks with SLMs. In *ICSE Workshops*. IEEE/ACM.
- Maria Trofimova, Anton Shevtsov, Badertdinov Ibragim, Konstantin Pyaev, Simon Karasik, and Alexander Golubev. 2025. Openhands trajectories with qwen3-coder-480b-a35b-instruct. *Nebius blog*.
- Jay Vaghasiya, Omkar Ghugarkar, Vishvesh Bhat, Vipul Dholaria, and Julian McAuley. 2025. CoreThink: A Symbolic Reasoning Layer to reason over Long Horizon Tasks with LLMs. *arXiv preprint arXiv:2509.00971*.
- Sergey Vakhreev. 2025. AI Coding Agent for Software Development - Refact.ai. <https://refact.ai/blog/2025/open-source-sota-on-swe-bench-verified-refact-ai/>. Accessed: 2026-01-15, from Refact.ai is now the #1 open-source AI Agent on SWE-bench.
- Konstantinos Vergopoulos, Mark Niklas Mueller, and Martin Vechev. 2025. Automated benchmark generation for repository-level coding tasks. In *ICLR 2025 Third Workshop on Deep Learning for Code*.
- Nguyen Phu Vinh, Anh Chung Hoang, Chris Ngo, and Truong-Son Hy. 2025. Repeton: Structured Bug Repair with ReAct-Guided Patch-and-Test Cycles. *arXiv preprint arXiv:2506.08173*.
- Boshi Wang, Weijian Xu, Yunsheng Li, Mei Gao, Yujia Xie, Huan Sun, and Dongdong Chen. 2025a. Improving Code Localization with Repository Memory. *arXiv preprint arXiv:2510.01003*.
- Chaozheng Wang, Zezhou Yang, Shuzheng Gao, Cuiyun Gao, Ting Peng, Hailiang Huang, Yuetang Deng, and Michael R. Lyu. 2025b. RAG or fine-tuning? A comparative study on LLMs-based code completion in industry. In *SIGSOFT FSE Companion*, pages 93–104. ACM.
- Haoran Wang, Zhenyu Hou, Yao Wei, Jie Tang, and Yuxiao Dong. 2025c. SWE-Dev: Building Software Engineering Agents with Training and Inference Scaling. *arXiv preprint arXiv:2506.07636*.
- Jinghui Wang, Shaojie Wang, Yinghan Cui, Xuxing Chen, Chao Wang, Xiaojiang Zhang, Minglei Zhang, Jiarong Zhang, Wenhao Zhuang, Yuchen Cao, Wankang Bao, Haimo Li, Zheng Lin, Huiming Wang, Haoyang Huang, Zongxian Feng, Zizheng Zhan, Ken Deng, Wen Xiang, and 8 others. 2025d. SeamlessFlow: A Trainer Agent Isolation RL Framework Achieving Bubble-Free Pipelines via Tag Scheduling. *arXiv preprint arXiv:2508.11553*.
- Junhao Wang, Daoguang Zan, Shulin Xin, Siyao Liu, Yurong Wu, and Kai Shen. 2025e. SWE-Mirror: Scaling Issue-Resolving Datasets by Mirroring Issues Across Repositories. *arXiv preprint arXiv:2509.08724*.
- Lilin Wang, Lucas Ramalho, Alan Celestino, Phuc Anthony Pham, Yu Liu, Umang Kumar Sinha, Andres Portillo, Onassis Osunwa, and Gabriel Maduekwe. 2025f. SWE-Bench++: A Framework for the Scalable Generation of Software Engineering Benchmarks from Open-Source Repositories. *arXiv preprint arXiv:2512.17419*.
- Qihao Wang, Ziming Cheng, Shuo Zhang, Fan Liu, Rui Xu, Heng Lian, Kunyi Wang, Xiaoming Yu, Jianghao Yin, Sen Hu, Yue Hu, Shaolei Zhang, Yanbing Liu, Ronghao Chen, and Huacan Wang. 2026. Mem-Govern: Enhancing Code Agents through Learning from Governed Human Experiences. *arXiv preprint arXiv:2601.06789*.
- Ruiyi Wang and Prithviraj Ammanabrolu. 2025. A practitioner’s guide to multi-turn agentic reinforcement learning. In *First Workshop on Multi-Turn Interactions in Large Language Models*.
- Weixun Wang, XiaoXiao Xu, Wanhe An, Fangwen Dai, Wei Gao, Yancheng He, Ju Huang, Qiang Ji, Hanqi Jin, Xiaoyang Li, Yang Li, Zhongwen Li, Shirong Lin, Jiashun Liu, Zenan Liu, Tao Luo, Dilxat Muhtar, Yuanbin Qu, Jiaqiang Shi, and 71 others. 2025g. Let It Flow: Agentic Crafting on Rock and Roll, Building the ROME Model within an Open Agentic Learning Ecosystem. *arXiv preprint arXiv:2512.24873*.
- Xincheng Wang, Pengfei Gao, Xiangxin Meng, Chao Peng, Ruida Hu, Yun Lin, and Cuiyun Gao. 2025h. Aegis: An agent-based framework for general bug reproduction from issue descriptions. In *Proceedings of the 33rd ACM International Conference on the Foundations of Software Engineering*, Trondheim, Norway. ACM. Industry Papers.

- Xingyao Wang, Boxuan Li, Yufan Song, Frank F. Xu, Xiangru Tang, Mingchen Zhuge, Jiayi Pan, Yueqi Song, Bowen Li, Jaskirat Singh, Hoang H. Tran, Fuqiang Li, Ren Ma, Mingzhang Zheng, Bill Qian, Yanjun Shao, Niklas Muennighoff, Yizhe Zhang, Binyuan Hui, and 5 others. 2025i. [Openhands: An open platform for AI software developers as generalist agents](#). In *The Thirteenth International Conference on Learning Representations*.
- Yanlin Wang, Yanli Wang, Daya Guo, Jiachi Chen, Ruikai Zhang, Yuchi Ma, and Zibin Zheng. 2025j. [RlCoder: Reinforcement learning for repository-level code completion](#). In *2025 IEEE/ACM 47th International Conference on Software Engineering (ICSE)*, page 1140–1152. IEEE.
- Yanlin Wang, Wanjun Zhong, Yanxian Huang, Ensheng Shi, Min Yang, Jiachi Chen, Hui Li, Yuchi Ma, Qianxiang Wang, and Zibin Zheng. 2025k. [Agents in software engineering: survey, landscape, and vision](#). *Automated Software Engineering*, 32(2).
- Yibo Wang, Zhihao Peng, Ying Wang, Zhao Wei, Hai Yu, and Zhiliang Zhu. 2025l. [MCTS-Refined CoT: High-Quality Fine-Tuning Data for LLM-Based Repository Issue Resolution](#). *arXiv preprint arXiv:2506.12728*.
- You Wang, Michael Pradel, and Zhongxin Liu. 2025m. [Are "Solved Issues" in SWE-bench Really Solved Correctly? An Empirical Study](#). *arXiv preprint arXiv:2503.15223*.
- Zora Zhiruo Wang, Akari Asai, Xinyan Velocity Yu, Frank F. Xu, Yiqing Xie, Graham Neubig, and Daniel Fried. 2025n. [CodeRAG-bench: Can retrieval augment code generation?](#) In *Findings of the Association for Computational Linguistics: NAACL 2025*, pages 3199–3214, Albuquerque, New Mexico. Association for Computational Linguistics.
- Yuxiang Wei, Olivier Duchenne, Jade Copet, Quentin Carbonneaux, LINGMING ZHANG, Daniel Fried, Gabriel Synnaeve, Rishabh Singh, and Sida Wang. 2025a. [SWE-RL: Advancing LLM reasoning via reinforcement learning on open software evolution](#). In *The Thirty-ninth Annual Conference on Neural Information Processing Systems*.
- Yuxiang Wei, Zhiqing Sun, Emily McMillin, Jonas Gehring, David Zhang, Gabriel Synnaeve, Daniel Fried, Lingming Zhang, and Sida Wang. 2025b. [Toward Training Superintelligent Software Agents through Self-Play SWE-RL](#). *arXiv preprint arXiv:2512.18552*.
- Sherman Wong, Zhenting Qi, Zhaodong Wang, Nathan Hu, Samuel Lin, Jun Ge, Erwin Gao, Wenlin Chen, Yilun Du, Minlan Yu, and Ying Zhang. 2025. [Confucius Code Agent: Scalable Agent Scaffolding for Real-World Codebases](#). *arXiv preprint arXiv:2512.10398*.
- W. Eric Wong, Ruizhi Gao, Yihao Li, Rui Abreu, and Franz Wotawa. 2016. [A survey on software fault localization](#). *IEEE Trans. Softw. Eng.*, 42(8):707–740.
- Di Wu, Wasi Uddin Ahmad, Dejiao Zhang, Murali Krishna Ramanathan, and Xiaofei Ma. 2024. [Repoformer: Selective retrieval for repository-level code completion](#). In *ICML*. OpenReview.net.
- Junde Wu. 2025. [Git Context Controller: Manage the Context of LLM-based Agents like Git](#). *arXiv preprint arXiv:2508.00031*.
- Chun Xia, Yinlin Deng, Soren Dunn, and Lingming Zhang. 2025a. [Demystifying llm-based software engineering agents](#). *Proceedings of the ACM on Software Engineering*, 2:801 – 824.
- Chunqiu Steven Xia, Zhe Wang, Yan Yang, Yuxiang Wei, and Lingming Zhang. 2025b. [Live-SWE-agent: Can Software Engineering Agents Self-Evolve on the Fly?](#) *arXiv preprint arXiv:2511.13646*.
- Yuan-An Xiao, Pengfei Gao, Chao Peng, and Yingfei Xiong. 2025. [Improving the Efficiency of LLM Agent Systems through Trajectory Reduction](#). *arXiv preprint arXiv:2509.23586*.
- Xiaomi-LLM-Core-Team, Bangjun Xiao, Bingquan Xia, Bo Yang, Bofei Gao, Bowen Shen, Chen Zhang, Chenhong He, Chiheng Lou, Fuli Luo, Gang Wang, Gang Xie, Hailin Zhang, Hanglong Lv, Hanyu Li, Heyu Chen, Hongshen Xu, Houbin Zhang, Huaqiu Liu, and 107 others. 2026. [MiMo-V2-Flash Technical Report](#). *arXiv preprint arXiv:2601.02780*.
- Chengxing Xie, Bowen Li, Chang Gao, He Du, Wai Lam, Difan Zou, and Kai Chen. [Swe-fixer: Training open-source llms for effective and efficient github issue resolution](#). In *ICLR 2025 Third Workshop on Deep Learning for Code*.
- Bojian Xiong, Yikun Lei, Xikai Liu, Shaowei Zhang, Pengyun Zhu, Yan Liu, Yongqi Leng, Ling Shi, Meizhi Zhong, Yurong Zhang, Yan Gao, Yiwu, Yao Hu, and Deyi Xiong. 2025. [Think-search-patch: A retrieval-augmented reasoning framework for repository-level code repair](#). In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing: Industry Track*, pages 1555–1566, Suzhou (China). Association for Computational Linguistics.
- Jingxuan Xu, Ken Deng, Weihao Li, Songwei Yu, Huaixi Tang, Haoyang Huang, Zhiyi Lai, Zizheng Zhan, Yanan Wu, Chenchen Zhang, Kepeng Lei, Yifan Yao, Xinpeng Lei, Wenqiang Zhu, Zongxian Feng, Han Li, Junqi Xiong, Dailin Li, Zuchen Gao, and 20 others. 2025a. [SWE-Compass: Towards Unified Evaluation of Agentic Coding Abilities for Large Language Models](#). *arXiv preprint arXiv:2511.05459*.
- Wendong Xu, Jing Xiong, Chenyang Zhao, Qiujiang Chen, Haoran Wang, Hui Shen, Zhongwei Wan, Jianbo Dai, Taiqiang Wu, He Xiao, Chaofan Tao, Z. Morley Mao, Ying Sheng, Zhijiang Guo, Hongxia Yang, Bei Yu, Lingpeng Kong, Quanguan Gu, and Ngai Wong. 2025b. [SwingArena: Competitive Programming Arena for Long-context GitHub Issue Solving](#). *arXiv preprint arXiv:2505.23932*.

- An Yang, Anfeng Li, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chang Gao, Chengen Huang, Chenxu Lv, Chujie Zheng, Dayiheng Liu, Fan Zhou, Fei Huang, Feng Hu, Hao Ge, Haoran Wei, Huan Lin, Jialong Tang, and 41 others. 2025a. Qwen3 Technical Report. *arXiv preprint arXiv:2505.09388*.
- Boyang Yang, Jiadong Ren, Shunfu Jin, Yang Liu, Feng Liu, Bach Le, and Haoye Tian. 2025b. Enhancing repository-level software repair via repository-aware knowledge graphs. *arXiv preprint arXiv:2503.21710*.
- John Yang, Carlos E Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. 2024. Swe-agent: Agent-computer interfaces enable automated software engineering. *Advances in Neural Information Processing Systems*, 37:50528–50652.
- John Yang, Carlos E Jimenez, Alex L Zhang, Kilian Lieret, Joyce Yang, Xindi Wu, Ori Press, Niklas Muennighoff, Gabriel Synnaeve, Karthik R Narasimhan, Diyi Yang, Sida Wang, and Ofir Press. 2025c. [SWE-bench multimodal: Do AI systems generalize to visual software domains?](#) In *The Thirteenth International Conference on Learning Representations*.
- John Yang, Kilian Lieret, Carlos E Jimenez, Alexander Wettig, Kabir Khandpur, Yanzhe Zhang, Binyuan Hui, Ofir Press, Ludwig Schmidt, and Diyi Yang. 2025d. [SWE-smith: Scaling data for software engineering agents](#). In *The Thirty-ninth Annual Conference on Neural Information Processing Systems Datasets and Benchmarks Track*.
- Xu Yang. 2026. Lingxi/docs/Lingxi Technical Report 2505.pdf at master · lingxi-agent/Lingxi. <https://github.com/lingxi-agent/Lingxi/blob/master/docs/Lingxi%20Technical%20Report%202505.pdf>. Accessed: 2026-01-15, from GitHub.
- Zonghan Yang, Shengjie Wang, Kelin Fu, Wenyang He, Weimin Xiong, Yibo Liu, Yibo Miao, Bofei Gao, Yejie Wang, Yingwei Ma, Yanhao Li, Yue Liu, Zhenxing Hu, Kaitai Zhang, Shuyi Wang, Huarong Chen, Flood Sung, Yang Liu, Yang Gao, and 2 others. 2025e. Kimi-Dev: Agentless Training as Skill Prior for SWE-Agents. *arXiv preprint arXiv:2509.23045*.
- Boxi Yu, Yuxuan Zhu, Pinjia He, and Daniel Kang. 2025a. UTBoost: Rigorous Evaluation of Coding Agents on SWE-Bench. *arXiv preprint arXiv:2506.09289*.
- Jiahao Yu, Zelei Cheng, Xian Wu, and Xinyu Xing. 2025b. Building Coding Agents via Entropy-Enhanced Multi-Turn Preference Optimization. *arXiv preprint arXiv:2509.12434*.
- Zhongming Yu, Hejia Zhang, Yujie Zhao, Hanxian Huang, Matrix Yao, Ke Ding, and Jishen Zhao. 2025c. [Orcaloca: An LLM agent framework for software issue localization](#). In *Forty-second International Conference on Machine Learning*.
- Karina Zainullina, Alexander Golubev, Maria Trofimova, Sergei Polezhaev, Ibragim Badertdinov, Daria Litvintseva, Simon Karasik, Filipp Fisin, Sergei Skvortsov, Maksim Nekrashevich, Anton Shevtsov, and Boris Yangel. 2025. Guided Search Strategies in Non-Serializable Environments with Applications to Software Engineering Agents. *arXiv preprint arXiv:2505.13652*.
- Daoguang Zan, Zhirong Huang, Wei Liu, Hanwu Chen, Shulin Xin, Linhao Zhang, Qi Liu, Aoyan Li, Lu Chen, Xiaojian Zhong, Siyao Liu, Yongsheng Xiao, Liangqiang Chen, Yuyu Zhang, Jing Su, Tianyu Liu, RUI LONG, Ming Ding, and liang xiang. 2025. [Multi-SWE-bench: A multilingual benchmark for issue resolving](#). In *The Thirty-ninth Annual Conference on Neural Information Processing Systems Datasets and Benchmarks Track*.
- Daoguang Zan, Zhirong Huang, Ailun Yu, Shaoxin Lin, Yifan Shi, Wei Liu, Dong Chen, Zongshuai Qi, Hao Yu, Lei Yu, Dezhi Ran, Muhao Zeng, Bo Shen, Pan Bian, Guangtai Liang, Bei Guan, Pengjie Huang, Tao Xie, Yongji Wang, and Qianxiang Wang. 2024. SWE-bench-java: A GitHub Issue Resolving Benchmark for Java. *arXiv preprint arXiv:2408.14354*.
- Aohan Zeng, Xin Lv, Qinkai Zheng, Zhenyu Hou, Bin Chen, Chengxing Xie, Cunxiang Wang, Da Yin, Hao Zeng, Jiajie Zhang, and 1 others. 2025a. glm-4.5: Agentic, reasoning, and coding (arc) foundation models. *arXiv preprint arXiv:2508.06471*.
- Guangtao Zeng, Maohao Shen, Delin Chen, Zhenting Qi, Subhro Das, Dan Gutfreund, David Cox, Gregory Wornell, Wei Lu, Zhang-Wei Hong, and Chuang Gan. 2025b. Satori-SWE: Evolutionary Test-Time Scaling for Sample-Efficient Software Engineering. *arXiv preprint arXiv:2505.23604*.
- Liang Zeng, Yongcong Li, Yuzhen Xiao, Changshi Li, Chris Yuhao Liu, Rui Yan, Tianwen Wei, Ju-jie He, Xuchen Song, Yang Liu, and Yahui Zhou. 2025c. Skywork-SWE: Unveiling Data Scaling Laws for Software Engineering in LLMs. *arXiv preprint arXiv:2506.19290*.
- Zizheng Zhan, Ken Deng, Jinghui Wang, Xiaojiang Zhang, Huaixi Tang, Minglei Zhang, Zhiyi Lai, Haoyang Huang, Wen Xiang, Kun Wu, Wenhao Zhuang, Shaojie Wang, Shangpeng Yan, Kepeng Lei, Zongxian Feng, Huiming Wang, Zheng Lin, Mengtong Li, Mengfei Xie, and 21 others. 2025. KAT-Coder Technical Report. *arXiv preprint arXiv:2510.18779*.
- Jenny Zhang, Shengran Hu, Cong Lu, Robert Lange, and Jeff Clune. 2025a. Darwin Godel Machine: Open-Ended Evolution of Self-Improving Agents. *arXiv preprint arXiv:2505.22954*.
- Jiwei Zhang, Jianxun Lian, Haiming Qin, Mingyang Zhou, KeZhong Lu, Rui Mao, and Hao Liao. 2025b. [Hierarchical reward modeling for fault localization in large code repositories](#). In *Findings of the Association for Computational Linguistics: EMNLP 2025*,

- pages 17782–17796, Suzhou, China. Association for Computational Linguistics.
- Kechi Zhang, Huangzhao Zhang, Ge Li, Jinliang You, Jia Li, Yunfei Zhao, and Zhi Jin. 2026. SEAlign: Alignment training for software engineering agent. In *ICSE*. ACM.
- Kexun Zhang, Weiran Yao, Zuxin Liu, Yihao Feng, Zhiwei Liu, Rithesh R N, Tian Lan, Lei Li, Renze Lou, Jiacheng Xu, Bo Pang, Yingbo Zhou, Shelby Heinicke, Silvio Savarese, Huan Wang, and Caiming Xiong. 2025c. *Diversity empowers intelligence: Integrating expertise of software engineering agents*. In *The Thirteenth International Conference on Learning Representations*.
- Kexun Zhang, Shang Zhou, Danqing Wang, William Yang Wang, and Lei Li. 2025d. *Scaling LLM inference efficiently with optimized sample compute allocation*. In *Proceedings of the 2025 Conference of the Nations of the Americas Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)*, pages 7959–7973, Albuquerque, New Mexico. Association for Computational Linguistics.
- Lei Zhang, Jiayi Yang, Min Yang, Jian Yang, Mouxian Chen, Jiajun Zhang, Zeyu Cui, Binyuan Hui, and Junyang Lin. 2025e. *Synthesizing software engineering data in a test-driven manner*. In *Forty-second International Conference on Machine Learning*.
- Linghao Zhang, Shilin He, Chaoyun Zhang, Yu Kang, Bowen Li, Chengxing Xie, Junhao Wang, Maoquan Wang, Yufan Huang, Shengyu Fu, Elsie Nallipogu, Qingwei Lin, Yingnong Dang, Saravan Rajmohan, and Dongmei Zhang. 2025f. *SWE-bench goes live!* In *The Thirty-ninth Annual Conference on Neural Information Processing Systems Datasets and Benchmarks Track*.
- Linhao Zhang, Daoguang Zan, Quanshun Yang, Zhirong Huang, Dong Chen, Bo Shen, Tianyu Liu, Yongshun Gong, Huang Pengjie, Xudong Lu, Guangtai Liang, Lizhen Cui, and Qianxiang Wang. 2025g. *CodeV: Issue resolving with visual data*. In *Findings of the Association for Computational Linguistics: ACL 2025*, pages 7350–7361, Vienna, Austria. Association for Computational Linguistics.
- Quanjin Zhang, Chunrong Fang, Yuxiang Ma, Weisong Sun, and Zhenyu Chen. 2023. *A survey of learning-based automated program repair*. *ACM Trans. Softw. Eng. Methodol.*, 33(2).
- Wenhao Zhang, Yuexiang Xie and Dawei Gao, Xuchen Pan, Yaliang Li, Bolin Ding, and Jingren Zhou. 2025h. SWE-Bench - AgentScope. <https://doc.agentscope.io/v0/en/tutorial/swe.html>. Accessed: 2026-01-15, from doc.agentscope.io.
- Yilin Zhang, Xinran Zhao, Zora Zhiruo Wang, Chenyang Yang, Jiayi Wei, and Tongshuang Wu. 2025i. cAST: Enhancing Code Retrieval-Augmented Generation with Structural Chunking via Abstract Syntax Tree. *arXiv preprint arXiv:2506.15655*.
- Yuntong Zhang, Haifeng Ruan, Zhiyu Fan, and Abhik Roychoudhury. 2024. *Autocoderover: Autonomous program improvement*. In *Proceedings of the 33rd ACM SIGSOFT International Symposium on Software Testing and Analysis, ISSTA '24*, page 1592–1604. ACM.
- Zejun Zhang, Jian Wang, Qingyun Yang, Yifan Pan, Yi Tang, Yi Li, Zhenchang Xing, Tian Zhang, Xuan-dong Li, and Guoan Zhang. 2025j. A Benchmark for Localizing Code and Non-Code Issues in Software Projects. *arXiv preprint arXiv:2509.25242*.
- Zhaoxi Zhang, Yitong Duan, Yanzhi Zhang, Yiming Xu, Weikang Li, Jiahui Liang, Deguo Xia, Jizhou Huang, Jiyan He, and Yunfang Wu. 2025k. One Tool Is Enough: Reinforcement Learning for Repository-Level LLM Agents. *arXiv preprint arXiv:2512.20957*.
- Songwen Zhao, Danqing Wang, Kexun Zhang, Jiaxuan Luo, Zhuo Li, and Lei Li. 2025. Is Vibe Coding Safe? Benchmarking Vulnerability of Agent-Generated Code in Real-World Tasks. *arXiv preprint arXiv:2512.03262*.
- Dewu Zheng, Yanlin Wang, Ensheng Shi, Xilin Liu, Yuchi Ma, Hongyu Zhang, and Zibin Zheng. 2024. Top General Performance = Top Domain Performance? DomainCodeBench: A Multi-domain Code Generation Benchmark. *arXiv preprint arXiv:2412.18573*.
- Jian Zhou, Hongyu Zhang, and David Lo. 2012. Where should the bugs be fixed? - more accurate information retrieval-based bug localization based on bug reports. In *Proceedings of the 34th International Conference on Software Engineering, ICSE '12*, page 14–24. IEEE Press.
- Xuhui Zhou, Valerie Chen, Zora Zhiruo Wang, Graham Neubig, Maarten Sap, and Xingyao Wang. 2025. TOM-SWE: User Mental Modeling For Software Engineering Agents. *arXiv preprint arXiv:2510.21903*.
- Yiqi Zhu, Apurva Gandhi, and Graham Neubig. 2025a. Training Versatile Coding Agents in Synthetic Environments. *arXiv preprint arXiv:2512.12216*.
- Yuxuan Zhu, Tengjun Jin, Yada Pruksachatkun, Andy Zhang, Shu Liu, Sasha Cui, Sayash Kapoor, Shayne Longpre, Kevin Meng, Rebecca Weiss, Fazl Barez, Rahul Gupta, Jwala Dhamala, Jacob Merizian, Mario Giulianelli, Harry Coppock, Cozmin Ududec, Jasjeet Sekhon, Jacob Steinhardt, and 6 others. 2025b. Establishing Best Practices for Building Rigorous Agentic Benchmarks. *arXiv preprint arXiv:2507.02825*.

A Appendix

A.1 Related work

Code generation. The application of LLMs in the programming domain has witnessed explosive growth. Early research focused primarily on function-level code generation, with benchmarks such as HumanEval (Chen et al., 2021) serving as standard metrics. However, generic benchmarks often fail to capture the nuances of real-world development. To bridge this gap, recent initiatives (Zheng et al., 2024; Li et al., 2025c; Bogomolov et al., 2024) have attempted to extend evaluation tasks to align more closely with realistic software development scenarios, revealing the limitations of general models in specialized domains. Concurrently, methods are also evolving to capture these broader contexts. While foundational approaches primarily relied on SFT (Liu et al., 2024a) or standard retrieval-augmented generation (Wu et al., 2024; Wang et al., 2025n), RL-based methods emerged as a pivotal direction for handling complex coding tasks (Wang et al., 2025j).

Automated software generation. The primary goal of this task is to autonomously construct complete and executable software systems starting from high-level natural language requirements. Unlike code completion, it necessitates covering the SDLC, including requirement analysis, system design, coding, and testing. To address the complexity and potential logic inconsistencies in this process, state-of-the-art frameworks like ChatDev (Qian et al., 2024) and MetaGPT (Hong et al., 2024) leverage multi-agent collaboration, simulating human development teams to decompose complex tasks into streamlined and verifiable workflows. Recently (Luo et al., 2025a) achieve repository-level generation from scratch. It introduces the Repository Planning Graph (RPG), which encodes file structures and data flows into a unified graph, effectively replacing free-form natural language with an explicit blueprint for consistent long-horizon planning.

Automated software maintenance. Issue resolution is intrinsically linked to the broader domain of automated software maintenance. Methodologies established in this field are frequently encapsulated as callable tools to augment the capabilities of LLMs in software development tasks. Key techniques utilized to enhance LLM performance include bug reproduction approaches (Khatib et al.,

2025; Kitsios et al., 2025), fault localization approaches such as SBFL (Wong et al., 2016; Zhou et al., 2012; Qi et al., 2022; Hu et al., 2014; Chakraborty et al., 2025; Chang et al., 2025; Zhang et al., 2025b), code search approaches (Zhang et al., 2025j), and test generation approaches (Ahmed et al., 2024; Mündler et al., 2024; Han et al., 2025). These tools provide agents with precise error locations, relevant code context, and verification mechanisms necessary for effective resolution. Furthermore, recent research has expanded the scope of automated software maintenance by utilizing issue resolution data to construct dialogue datasets that capture real-world human-computer collaboration (Garg et al., 2025), and critically examining the security risks inherent in agent-generated code through vulnerability benchmarking (Zhao et al., 2025).

Automated environment setup. Recent initiatives focus on automating the configuration of runtime environments for entire repositories (Eliseeva et al., 2025; Kovrigin et al., 2025; Vergopoulos et al., 2025). This capability develops in parallel with data construction for issue resolution.

Related surveys. Existing surveys primarily focus on code generation (Jiang et al., 2025a; Wang et al., 2025b) or other tasks within the software engineering domain (Huang et al., 2023; Zhang et al., 2023; Guo et al., 2025a; Wang et al., 2025k). This paper bridges this gap by offering the first systematic survey dedicated to the entire spectrum of issue resolution, ranging from non-agent approaches to the latest agentic advancements.

A.2 Detailed discussions on background

The issue resolution task aims to automatically resolve reported issues. As illustrated in Figure 1, it is formally defined by an instance represented as a triple $\mathcal{I} = (\mathcal{D}, \mathcal{C}, \mathcal{T})$, where these components map directly to the benchmark’s metadata structure:

- $\mathcal{D}$: Issue description, which is the original GitHub issue text description detailing the bug or feature request.
- $\mathcal{C}$: Codebase, the collection of source files at the specific commit state before the issue was resolved. This state is precisely defined by the repository identifier (owner/repo) and a base commit hash, requiring the evaluation environment to perform a git clone and subsequent git checkout to establish the exact pre-fix version.

- $\mathcal{T}$: Tests, the complete set of unit and system tests associated with the issue, derived from the original developer’s test patch. This set is explicitly categorized into two subsets: $T_{\text{fail} \rightarrow \text{pass}}$, containing tests that fail on C and must pass after the patch (verifying the fix), and $T_{\text{pass} \rightarrow \text{pass}}$, containing tests that must pass both before and after (ensuring no regression).

Given only the inputs $(\mathcal{D}, C)$, the model proposes an edit to resolve the issue. The model’s output is a Patch $\mathcal{P}$ (represented as a patch file). (The target solution is the gold patch provided in the dataset, which is not revealed to the model). The patch $\mathcal{P}$ must be successfully applied to C using a standard patch application utility. The codebase resulting from this application is C' . Crucially, the tests $\mathcal{T}$ are not revealed to the model during this process.

Evaluation. To evaluate a proposed solution (patch $\mathcal{P}$), it is first applied to the original codebase C to obtain C' . The repository’s test suite $\mathcal{T}$ is then executed on C' . The evaluation framework captures the output in a test log, which is subsequently processed by a log parser to determine the test results (e.g., pass or fail). Based on the parsed results, the solution is scored: it is considered successful if the patch applies correctly and all tests in $\mathcal{T}$ pass. The final benchmark metric is the Resolve Rate, defined as the percentage of tasks that are successfully resolved.

Data Construction. This process is structured into four main stages.

First, (1) *Repo Selection and Data Scraping* involves collecting a large set of PRs from popular, well-maintained open-source repositories (e.g., 12 popular Python repositories for the original SWE-bench).

Second, (2) *Attribute-based Filtering* narrows down the candidates, selecting only merged PRs that are documented to resolve a specific GitHub issue and that make modifications to the repository’s test files (indicating that tests were contributed).

Third, (3) *Execution-based Filtering* is a critical stage that ensures tasks are reproducible and valid. To guarantee reliable environment construction, recent works increasingly leverage CI/CD configurations—specifically GitHub Actions workflows found under `.github/workflows/`—as the ground truth for dependency management. For instance, Multi-SWE-bench (Zan et al., 2025) ex-

tracts build steps from these workflows to create isolated Dockerized environments. Similarly, SWE-Sharp-Bench (Mhatre et al., 2025) confirms build viability by executing local GitHub Actions workflows, ensuring the repository successfully builds and passes tests at the latest commit. By filtering out instances that fail these automated installation or runtime checks, this stage establishes a stable foundation. Finally, it verifies the PR’s corrective nature by executing the test suite before and after applying the patch, retaining only instances that demonstrate at least one clear Fail-to-Pass (F2P) test transition.

Finally, (4) *Manual Verification* is performed to ensure the quality and usability of the filtered tasks. This step often involves human inspection to check for the clarity of the problem description (issue), the self-contained nature of the task, and its overall suitability for the benchmark.

While this initial pipeline was effective for creating a static dataset (e.g., the 2,294 SWE-bench instances), its reliance on a complex, manual-heavy environment setup (especially in stages 3 and 4) and its susceptibility to data contamination limited its long-term scalability and dynamism. Consequently, subsequent research has focused on enhancing these data methods, leading to two major axes of improvement: Data Collection for building more dynamic benchmarks, and Data Synthesis for creating high-quality synthetic datasets for training large language models.

A.3 Detailed discussions on data

Table 1 provides a comprehensive survey of the rapidly evolving domain of SWE-bench related datasets. While the original SWE-bench and its refined iterations (such as SWE-bench Lite and SWE-bench Verified) set the standard for Python-based issue resolution, recent works have significantly expanded the scope of evaluation along three main axes: language diversity, modality, and scale.

Multilingual expansion. A major limitation of early benchmarks was their exclusive focus on Python. To address this, datasets such as SWE-bench-java, Multi-SWE-bench, SWE-bench Multilingual, and SWE-PolyBench have been introduced. These datasets extend evaluation capabilities to a wide array of popular languages including Java, C++, Go, Rust, JavaScript, and TypeScript, encompassing thousands of repositories to test the generalization ability of agents across different syntaxes

Dataset	Language	Multimodal	Repos	Amount	Environment	Link
Single-PL Datasets						
SWE-Fixer	Python	✗	856	115,406	✗	🤖🤖🤖
SWE-smith	Python	✗	128	50k	✓	🤖🤖🤖
SWE-Lego	Python	✗	3,251	32,119	✓	🤖🤖🤖
SWE-rebench	Python	✗	3,468	21,336	✓	🤖🤖🤖
SWE-bench-train	Python	✗	37	19k	✗	🤖🤖🤖
SWE-Flow	Python	✗	74	18,081	✓	🤖
Skywork-SWE	Python	✗	2,531	10,169	✓	/
R2E-Gym	Python	✗	10	8,135	✓	🤖🤖🤖
RepoForge	Python	✗	/	7.3k	✓	/
SWE-bench-extra	Python	✗	2k	6.38k	✓	🤖🤖🤖
SWE-Gym	Python	✗	11	2,438	✓	🤖🤖🤖
SWE-bench	Python	✗	12	2,294	✓	🤖🤖🤖
SWE-bench-java	Java	✗	19	1,797	✓	🤖🤖🤖
FEA-bench	Python	✗	83	1,401	✓	🤖🤖🤖
SWE-bench-Live	Python	✗	164	1,565	✓	🤖🤖🤖
Loc-Bench	Python	✗	/	560	✗	🤖🤖🤖
SWE-bench Verified	Python	✗	/	500	✓	🤖🤖🤖
SWE-bench Lite	Python	✗	12	300	✓	🤖🤖🤖
SWE-MERA	Python	✗	200	300	✓	🤖🤖🤖
SWE-Bench-CL	Python	✗	8	273	✓	🤖🤖🤖
SWE-Sharp-Bench	C#	✗	17	150	✓	🤖🤖🤖
SWE-Perf	Python	✗	12	140	✓	🤖🤖🤖
Visual SWE-bench	Python	✓	11	133	✓	🤖🤖🤖
SWE-EVO	Python	✗	7	48	✓	🤖
Multi-PL Datasets						
SWE-Mirror	Python, Rust, Go	✗	40	60k	✓	/
Multi-SWE-bench	Java, JS, TS, Go, Rust, C, C++	✗	76	4,723	✓	🤖🤖🤖
Swing-Bench	Python, Go, C++, Rust	✗	400	2300	✓	/
SWE-PolyBench	Python, Java, JS, TS	✗	21	2,110	✓	🤖🤖🤖
SWE-Compass	Python, JS, TS, Java, C, C++, Go, Rust, Kotlin, C#	✗	/	2,000	✓	🤖🤖🤖
SWE-Bench Pro	Python, Go, TS	✗	41	1,865	✓	🤖🤖🤖
SWE-bench++	Python, Go, TS, JS, Ruby, PHP, Java, Rust, C++, C#, C	✗	3,971	1,782	✓	🤖🤖🤖
SWE-Lancer	JS, TS	✗	/	1,488	✓	🤖🤖🤖
OmniGIRL	Python, TS, Java, JS	✓	15	959	✓	🤖🤖🤖
SWE-bench Multimodal	JS, TS, HTML, CSS	✓	17	619	✓	🤖🤖🤖
SWE-fficiency	Python, Cython	✗	9	498	✓	🤖🤖🤖
SWE-Factory	Python, Java, JS, TS	✗	12	430	✓	🤖🤖🤖
SWE-bench-Live-MultiLang & Windows	Python, JS, TS, C, C++, C#, Java, Go, Rust	✗	238	418	✓	🤖🤖🤖
SWE-bench Multilingual	C, C++, Go, Java, JS, TS, Rust, Python, Ruby, PHP	✗	42	300	✓	🤖🤖🤖
SWE-InfraBench	Python, TS	✗	/	100	✓	/

Table 1: A comprehensive survey and statistical overview of issue resolution datasets. We categorize these datasets based on programming language, modality support, source repositories, data scale (Amount), and the availability of reproducible execution environments.

and ecosystems.

Multimodality. Recognizing that modern software development often involves visual elements (particularly in frontend development), datasets like SWE-bench Multimodal, CodeV, and OmniGIRL have incorporated visual contexts. These benchmarks require agents to process not only code and text but also visual data, targeting languages like HTML, CSS, and TypeScript.

Scale and training resources. To support the training of more robust agents, several large-scale datasets have been curated. SWE-Smith, SWE-Fixer, and SWE-bench-extra dramatically increase

the volume of data, offering up to 115k instances (in the case of SWE-Fixer). Notably, as indicated in the "Environment" column, the field is shifting towards execution-based validation; unlike earlier static datasets, the majority of recent benchmarks now provide reproducible dockerized environments to rigorously verify agent generated patches.

A.4 Detailed discussions on training methods

Table 3 provides a comprehensive overview of recent SFT-based approaches.

Table 4 categorizes specialized models that have undergone further alignment, predominantly via Reinforcement Learning. The table sorts models by parameter size, revealing that smaller, dense

Dataset	Language	Repos	Amount	Link
R2E-Gym	Python	10	3,321	 
SWE-Gym	Python	11	491	 
SWE-Synth	Python	11	3,018	 
SWE-Fixer	Python	856	69,752	 
SWE-Factory	Python	10	2,809	 
SWE-rebench	Python	1,823	67,074	
SWE-Lego	Python	3251	14.6k	

Table 2: A survey of trajectory datasets used for agent training or analysis. We list the programming language, number of source repositories, and total trajectories for each dataset.

Model Name	Base Model	Size	Arch.	Training Scaffold	Res.(%)	Code	Data	Model
SWE-rebench-openhands-Qwen3-235B-A22B	Qwen3-235B-A22B	235B-A22B	MoE	OpenHands	59.9	/		
SWE-Lego-Qwen3-32B	Qwen3-32B	32B	Dense	OpenHands	57.6			
SWE-rebench-openhands-Qwen3-30B-A3B	Qwen3-30B-A3B	30B-A3B	MoE	OpenHands	49.7	/		
Devstral	Mistral Small 3	22B	Dense	OpenHands	46.8	/		
Co-Patcher	Qwen2.5-Coder-14B	3×14B	Dense	PatchPilot-mini	46.0		/	
SWE-Swiss-32B	Qwen2.5-32B-Instruct	32B	Dense	Agentless	45.0			
SWE-Lego-Qwen3-8B	Qwen3-8B	8B	Dense	OpenHands	44.4			
Lingma SWE-GPT	Qwen2.5-72B-Instruct	72B	Dense	SWESynInfer	30.2		/	/
SWE-Gym-Qwen-32B	Qwen2.5-Coder-32B	32B	Dense	OpenHands, MoatlessTools	20.6		/	
Lingma SWE-GPT	Qwen2.5-Coder-7B	7B	Dense	SWESynInfer	18.2		/	/
SWE-Gym-Qwen-14B	Qwen2.5-Coder-14B	14B	Dense	OpenHands, MoatlessTools	16.4		/	
SWE-Gym-Qwen-7B	Qwen2.5-Coder-7B	7B	Dense	OpenHands, MoatlessTools	10.6		/	

Table 3: Overview of SFT-based methods for issue resolution. This table categorizes models by their base architecture and training scaffold (Sorted by Performance).

models (e.g., 7B-32B) can achieve competitive performance against larger baselines when optimized with domain-specific rewards. The trends observed in the Reward column align with the design principles detailed in Section 4.2.2. While sparse outcome rewards based on test verification constitute the predominant approach, the data reveals a growing integration of process rewards. These dense feedback signals prove critical for stabilizing the training of smaller models during long-horizon tasks, thereby addressing the sparse signal challenges associated with complex repository-level debugging. Additionally, the heterogeneity observed in the Training Scaffolds category indicates a tendency to deploy RL atop established agent frameworks to enhance decision-making policies.

Finally, Table 5 lists general-purpose foundation models evaluated on issue resolution (Zhan et al., 2025; Seed et al., 2025; DeepSeek-AI et al., 2025; Team et al., 2025a; Yang et al., 2025a; Zeng et al.,

2025a; Agarwal et al., 2025; Chen et al., 2025a; Meituan-LongCat-Team et al., 2025; Xiaomi-LLM-Core-Team et al., 2026). In contrast to the specialized models above, these systems rely entirely on external inference scaffolds to bridge the gap between general reasoning and repository-level engineering. This comparison serves as a control group, highlighting the specific performance gains attributable to the SFT and RL pipelines described in Section 4.2.

A.5 Detailed discussions on applications

The evolution of AI in software engineering is characterized by four distinct stages of increasing autonomy and architectural sophistication:

Stage 1: Developer augmentation. The initial phase is dominated by AI pair programmers, such as GitHub Copilot, which integrate directly into Integrated Development Environments (IDEs). These tools focus on real-time code completion and sug-

gestions. Empirical evidence from enterprise adoption, such as at Accenture, indicates that these assistants can increase developer productivity by up to 55%, leading to widespread implementation at major technology firms like Shopify.

Stage 2: Workflow automation. The subsequent level introduces AI junior developers, represented by tools like Sweep AI. These agents automate the asynchronous lifecycle of software maintenance. Unlike isolated code completions, they autonomously parse GitHub issues, analyze the relevant codebase, generate necessary modifications, and submit complete pull requests for human review, effectively handling routine engineering tasks without constant supervision.

Stage 3: End-to-end autonomy. The most advanced operational stage involves fully autonomous agents like Devin. Functioning within secure sandboxed environments equipped with a shell, editor, and browser, these agents handle complex multi-step engineering tasks ranging from planning to execution. Their industrial impact is significant; for instance, during a major code migration at the fintech company Nubank, such autonomous agents reportedly achieved a 12x improvement in engineering efficiency compared to traditional methods.

Stage 4: Ecosystem integration. The latest trend emphasizes platform interoperability and compliance. Emerging tools like Claude Code focus on embedding AI capabilities into existing enterprise workflows with rigorous security governance. Concurrently, platforms like Trae utilize the Model Context Protocol (MCP) to orchestrate multi-agent architectures, fostering an extensible ecosystem where diverse AI tools can collaborate seamlessly. However, such protocols face scalability challenges, particularly the context overhead from storing cumulative tool-invocation results. Addressing this requires protocol-level optimizations such as Anthropic's MCP code execution to minimize the verbosity of feedback loops and maintain system efficiency.

Model Name	Base Model	Size	Arch.	Train. Scaffold	Reward	Res.(%)	Code	Data	Model
560B Models (MoE)									
LongCat-Flash-Think	LongCatFlash-Base	560B-A27B	MoE	R2E-Gym	Outcome	60.4	🔄	/	🤔
72B Models									
Kimi-Dev	Qwen 2.5-72B-Base	72B	Dense	BugFixer + TestWriter	Outcome	60.4	🔄	/	🤔
Multi-turn RL(Nebius)	Qwen2.5-72B-Instruct	72B	Dense	SWE-agent	Outcome	39.0	/	/	/
Agent-RLVR-RM-72B	Qwen2.5-Coder-72B	72B	Dense	Localization + Repair	Outcome	27.8	/	/	/
Agent-RLVR-72B	Qwen2.5-Coder-72B	72B	Dense	Localization + Repair	Outcome	22.4	/	/	/
70B Models									
SWE-RL	Llama-3.3-70B-Instruct	70B	Dense	Agentless-mini	Outcome	41.0	🔄	/	/
36B Models									
FoldAgent	Seed-OSS-36B-Instruct	36B	Dense	FoldAgent	Process	58.0	🔄	🌐	/
32B Models									
OpenHands Critic	Qwen2.5-Coder-32B	32B	Dense	SWE-Gym	/	66.4	🔄	/	🤔
KAT-Dev-32B	Qwen3-32B	32B	Dense	/	/	62.4	/	/	🤔
SWE-Swiss-32B	Qwen2.5-32B-Instruct	32B	Dense	/	Outcome	60.2	🔄	🤔	🤔
SeamlessFlow-32B	Qwen3-32B	32B	Dense	SWE-agent	Outcome	45.8	🔄	/	/
DeepSWE	Qwen3-32B	32B	Dense	R2E-Gym	Outcome	42.2	🔄	🤔	🤔
SA-SWE-32B	/	32B	Dense	SkyRL-Agent	/	39.4	/	/	/
OpenHands LM v0.1	Qwen2.5-Coder-32B	32B	Dense	SWE-Gym	/	37.2	🔄	/	🤔
SWE-Dev-32B	Qwen2.5-Coder-32B	32B	Dense	OpenHands	Outcome	36.6	🔄	/	🤔
Satori-SWE	Qwen2.5-Coder-32B	32B	Dense	Retriever + Code editor	Outcome	35.8	🔄	🤔	🤔
SoRFT-32B	Qwen2.5-Coder-32B	32B	Dense	Agentless	Outcome	30.8	/	/	/
Agent-RLVR-32B	Qwen2.5-Coder-32B	32B	Dense	Localization + Repair	Outcome	21.6	/	/	/
14B Models									
Agent-RLVR-14B	Qwen2.5-Coder-14B	14B	Dense	Localization + Repair	Outcome	18.0	/	/	/
SEAlign-14B	Qwen2.5-Coder-14B	14B	Dense	OpenHands	Process	17.7	/	/	/
9B Models									
SWE-Dev-9B	GLM-4-9B	9B	Dense	OpenHands	Outcome	13.6	🔄	/	🤔
8B Models									
SeamlessFlow-8B	Qwen3-8B	8B	Dense	SWE-agent	Outcome	27.4	🔄	/	/
SWE-Dev-8B	Llama-3.1-8B	8B	Dense	OpenHands	Outcome	18.0	🔄	/	🤔
7B Models									
SWE-Dev-7B	Qwen2.5-Coder-7B	7B	Dense	OpenHands	Outcome	23.4	🔄	/	🤔
SoRFT-7B	Qwen2.5-Coder-7B	7B	Dense	Agentless	Outcome	21.4	/	/	/
SEAlign-7B	Qwen2.5-Coder-7B	7B	Dense	OpenHands	Process	15.0	/	/	/

Table 4: A comprehensive overview of specialized models for issue resolution, categorized by parameter size. The table details each model’s base architecture, the training scaffold used for rollout, the type of reward signal employed (Outcome vs. Process), and their performance results (Res. %) on issue resolution benchmarks.

Model Name	Size	Arch.	Inf. Scaffold	Reward	Res.(%)	Code	Model
MiMo-V2-Flash	309B-A15B	MoE	Agentless	Outcome	73.4	🔄	🤔
KAT-Coder	/	/	Claude Code	Outcome	73.4	/	🌐
Deepseek V3.2	671B-A37B	MoE	Claude Code, RooCode	/	73.1	🔄	🤔
Kimi-K2-Instruct	1T	MoE	Agentless	Outcome	71.6	/	🤔
Qwen3-Coder	480B-A35B	MoE	OpenHands	Outcome	69.6	🔄	🤔
GLM-4.6	355B-A32B	MoE	OpenHands	Outcome	68.0	/	🤔
gpt-oss-120b	116.8B-A5.1B	MoE	Internal tool	Outcome	62.0	🔄	🤔
Minimax M2	230B-10B	MoE	R2E-Gym	Outcome	61.0	🔄	🤔
gpt-oss-20b	20.9B-A3.6B	MoE	Internal tool	Outcome	60.0	🔄	🤔
GLM-4.5-Air	106B-A12B	MoE	OpenHands	Outcome	57.6	/	/
Minimax M1-80k	456B-A45.9B	MoE	Agentless	Outcome	56.0	🔄	🌐
Minimax M1-40k	456B-A45.9B	MoE	Agentless	Outcome	55.6	🔄	🌐
Seed1.5-Thinking	200B-A20B	MoE	/	Outcome	47.0	🔄	/
Llama 4 Maverick	400B-A17B	MoE	mini-SWE-agent	Outcome	21.0	🔄	🤔
Llama 4 Scout	109B-17B	MoE	mini-SWE-agent	Outcome	9.1	🔄	🤔

Table 5: Overview of general foundation models evaluated on issue resolution. The table details the specific inference scaffolds (e.g., OpenHands, Agentless) employed during the evaluation process to achieve the reported results.